A S G D E T A I L E D D E S I G N

面向大模型智能体的数据结构详细设计说明书

Agent 状态图（ASG）V1.0 语义内核与执行闭环

> 依据文档：《面向大模型智能体的数据结构研究与设计报告》
>
> 文档版本：V1.0（详细设计）
>
> 编制日期：2026 年 10 月 24 日
>
> 适用阶段：阶段 A 语义内核 / 阶段 B 执行闭环

摘　要

本说明书依据《面向大模型智能体的数据结构研究与设计报告》（2026-10-24）开展详细设计，将其中提出的 Agent 状态图（ASG）抽象数据类型细化为可实施的工程规格。内容覆盖：模块划分与信任边界；十一种类型化记录的字段级数据结构定义与关系边类型系统；PostgreSQL 物理模式（含实现请求去重、预算约束与不可变性的具体约束）；observe、assess、revise、invalidate、prepare_effect、dispatch、resolve_effect、project_context、retire_or_purge 九个核心操作的接口契约；效果、尝试、预留、授权、任务五组状态机的转移表；串行提交、支持电路求值、失效传播与读取闸门、上下文投影、原子预留、outbox 执行、对账、删除传播与确定性回放九个核心算法；并发事务边界与不变量 I1 至 I10 的机制映射；授权规范化摘要、权限检查点、数据标签传播与删除域设计；错误码目录、复杂度边界、对抗测试用例集与阶段 A 至 E 实施计划。

本说明书实现范围对应研究设计报告的阶段 A（语义内核）与阶段 B（执行闭环），并为阶段 C（证据与上下文）预留接口；不包含模型选型、检索算法本体、跨工作空间分布式事务与差分隐私机制。全部设计遵循研究设计报告的边界声明：历史日志是系统接纳记录的依据而非外部真值证明；投影带水位并以纪元闸门约束关键读取；外部效果一律经执行网关并区分失败与未知；删除以墓碑门控阻止复活。本文不含任何虚构的实验结果或性能数字。

关键词：Agent 状态图；证据依赖；失效传播；执行恢复；授权绑定；资源预留上下文清单；确定性回放

目　录

1 引言.....

1 引言.....1

1.1 编写目的.....2

1.2 背景.....2

1.3 术语、缩略语与记号.....2

1.4 参考资料.....3

1.5 设计范围与边界.....3

2 总体设计.....3

2.1 设计原则.....4

2.2 模块划分与职责.....4

2.3 信任边界.....4

2.4 核心抽象与实现的映射.....5

2.5 技术选型与部署约束.....5

3 数据结构详细设计.....5

3.1 标识、摘要与基本类型.....6

3.2 记录通用信封 Record.....6

3.3 提交 Commit 与快照 Snapshot.....6

3.4 证据类记录 Observation.....7

3.5 主张类记录 Claim.....9

3.6 支持关系记录 Justification.....9

3.7 评估记录 Assessment.....11

3.8 工作类记录 TaskRevision.....11

3.9 动作意图 ActionIntent.....12

3.10 授权记录 AuthorizationGrant.....12
3.11 尝试 Attempt、回执 Receipt 与产物 Artifact.....12
3.12 上下文清单 ContextManifest.....13
3.13 关系边类型系统 typed_refs.....13
3.14 状态与枚举字典.....14
3.15 schema 演进规则.....14
4 数据库物理设计.....14
4.1 表清单与一致性要求.....15
4.2 核心表 DDL.....15
4.3 索引设计.....41
4.4 约束与不变量的第一道防线.....42
4.5 增长控制与归档.....42
5 操作接口详细设计.....42
5.0 通用约定.....42
5.1 observe：记录观察.....43
5.2 assess：主张评估.....44
5.3 revise：版本修订.....46
5.4 invalidate：失效传播.....47
5.5 prepare_effect：意图准备.....49
5.6 dispatch / execute_outbox：网关执行.....50
5.7 resolve_effect：对账与结算.....52
5.8 project_context：上下文投影.....53
5.9 retire_or_purge：撤回与删除.....54
5.10 辅助接口.....55

6 状态机详细设计.....55
6.1 效果状态机 EffectState.....55
6.2 尝试状态机与幂等契约.....56
6.3 资源预留状态机 ReservationState.....56
6.4 授权生命周期 GrantStatus.....56
6.5 任务状态机 TaskStatus.....57
7 核心算法设计.....57
7.1 串行提交协议.....57
7.2 支持电路求值.....60
7.3 失效传播与读取闸门.....63
7.4 上下文投影.....66
7.5 原子预留与结算.....68
7.6 outbox 执行循环.....71
7.7 幂等与对账.....74
7.8 删除传播与复活阻止.....76
7.9 确定性回放恢复.....78
8 并发控制与一致性设计.....80
8.1 工作空间内有序提交.....80
8.2 事务边界一览.....80
8.3 预算账本与写偏斜防范.....80
8.4 分支 fork 与 merge 协议.....80
8.5 不变量到机制的映射.....81
9 授权、隐私与删除设计.....81
9.1 operation_digest 规范化规则.....81

9.2 权限检查点 82
9.3 数据标签传播 82
9.4 删除域与能力边界 82
9.5 标识与摘要暴露控制 83
10 错误处理设计 83
11 性能与复杂度设计 83
11.1 操作复杂度（成立条件必须随行） 83
11.2 瓶颈与对策 84
12 测试与验证设计 84
12.1 三层评估框架 84
12.2 对抗场景测试用例 84
12.3 指标与门控 85
12.4 公平基线与消融 85
13 实施计划 86
附录 A 记录类型注册表 86
附录 B 需求追溯矩阵 86

注：本目录由域代码生成。编辑文档后如需刷新页码，请在目录上右键选择更新域并更新整个目录。

1 引言

1.1 编写目的

本说明书依据《面向大模型智能体的数据结构研究与设计报告》（2026 年 10 月 24 日版，下称研究设计报告）开展详细设计。研究设计报告已经完成问题定义、文献边界调研、结构选型与形式化定义，并给出操作契约、不变量与验证方案的框架；本说明书将其中的抽象数据类型 ASG（Agent 状态图）细化为可直接进入编码阶段的工程规格，包括模块划分、字段级数据结构定义、数据库物理模式、九个核心操作的接口契约、状态机转移表、核心算法伪代码、并发与一致性机制、授权与删除设计、错误码目录、性能边界以及测试与实施计划。

本说明书的预期读者为：语义内核与存储模块的开发工程师、执行网关开发工程师、平台评审人员、测试设计人员以及后续参与阶段 C 至阶段 E 扩展的研发成员。阅读本说明书前应先通读研究设计报告，本说明书不重复其论证过程，仅在必要处引用其结论与编号（如需求 R1–R9、不变量 I1–I10、来源标识 S1–S24 沿用研究报告附录 B 的定义）。

1.2 背景

长期运行的大模型智能体需要在模型之外的持久状态中区分：收到了什么（观察）、据此相信什么（主张）、准备做什么（任务与意图）、被允许做什么（授权）、实际做成了什么（尝试、回执与产物）。研究设计报告论证了将这些信息压入聊天历史或单一摘要不可行，并提出以带版本与证据依赖的 Agent 状态图（ASG）作为基础抽象数据类型：以不可变的类型化记录和事务提交历史为基础，当前知识视图、检索索引与工作上下文由这些记录投影生成。

研究设计报告同时明确了该设计的适用边界：ASG 服务于证据频繁变化、任务长期运行、多智能体共享状态或外部操作结果必须审计的工作负载；对于单次问答、无外部写操作且历史很短的应用，消息列表加显式任务状态即可满足。本说明书全部设计决策均以该边界为前提，不扩大承诺范围。

1.3 术语、缩略语与记号

本说明书使用的核心术语如下表，记号与研究报告第 4 章保持一致。

表 1 术语与缩略语

术语	含义
ASG	Agent 状态图，本设计的抽象数据类型总称，由记录集合、提交历史与投影构成
工作空间 workspace	提交历史与授权的作用域，多工作空间可并行，MVP 内单工作空间为串行提交
提交 commit	一批不可变记录及其确定性状态转换的原子单元，按序列号有序
记录 record	不可变的类型化事实单元，含通用信封与按 kind 区分的载荷
逻辑对象 object	跨版本稳定的实体标识，其某个确定版本由记录 ID 指向
投影 projection	由提交历史推导的当前状态视图，含当前版本表、任务状态、索引与上下文
reducer	固定版本的纯状态转换函数，回放提交历史以重建投影
分支 branch	自某快照 fork 出的比较空间，共享不可变记录，新增记录仅本分支可见
验证纪元 validation_epoch	单调递增整数，失效传播事务推进，读取闸门用于判断投影是否落后
策略纪元 policy_epoch	访问策略版本号，删除与权限变更时推进，旧快照不得绕过
绑定单元 binding unit	来源与其限制条件（否定条件、单位、时间、权限标签）的整体，投影时不可拆散
水位 applied_sequence	异步索引已应用的提交序号，与验证纪元共同标注索引新鲜度
效果状态 effect state	外部动作的七态外加 PARTIAL 的生命周期状态（见第 6 章）
预留 reservation	对共享资源的原子占用，状态含 reserved、uncertain、committed、released
执行网关 gateway	全部外部效果的唯一可信出口，负责重验证、尝试记录与回执持久化
连接器 connector	外部系统适配器，须声明幂等、查询、最终性与条件更新能力契约
规范化摘要 operation_digest	对操作、目标、主体、连接器、参数与产物内容按固定规则规范化后的摘要

术语	含义
删除域 deletion domain	受删除策略约束的内容集合：正文、嵌入、摘要、完整输入、附件、缓存及必要元数据
墓碑 tombstone	先于清理持久化的禁止读取与禁止恢复标记，阻止旧快照复活已删内容
回放 replay	以确定性 reducer 在已记录输入上重建状态的过程，不重新调用模型或外部工具
OUTCOME_UNKNOWN	无法判断外部操作是否生效的效果状态，须保留资源与授权占用
outbox	与意图准备同事务写入的待发送动作队列，由网关领取执行
Eligible	证据在指定查询范围、时间与策略下可被使用且相关验证尚有效的资格判定

1.4 参考资料

本说明书的直接输入为研究设计报告，其引用的一手来源（S1–S24）不在本文重复列出；下列资料在本说明书中被直接引用，标识沿用研究报告附录 B。

表 2 参考资料

标识	资料	用途
[SD]	面向大模型智能体的数据结构研究与设计报告，2026-10-24	本说明书的需求与概念设计来源
[S21]	RFC 8785 JSON Canonicalization Scheme，2020-06	operation_digest 规范化的基础（第 9.1 节）
[S20]	RFC 9110 HTTP Semantics，2022-06	幂等语义与条件请求的参照（第 7.7 节）
[S19]	PostgreSQL 18 官方文档，事务隔离章节	串行化与快照隔离差异的依据（第 8 章）
[S16]	W3C PROV-DM，2013-04-30	实体、活动、派生、修订与失效的关系语义来源
[S15]	A2A 协议规范 1.0.0，第 7.6.4 节	授权须绑定具体操作内容的外部佐证
[S14]	MCP 规范，2026-07-28 固定版	连接器适配边界的参照（第 2.6 节）
[S12]	Temporal 官方文档，事件历史与活动定义	执行恢复与幂等契约的工程先例（第 6、7 章）

1.5 设计范围与边界

实现范围：研究设计报告第 14 章阶段 A（语义内核）与阶段 B（执行闭环）的全部机制，即记录与提交模型、投影与失效传播、预算与授权账本、意图准备与 outbox 执行、UNKNOWN 对账、上下文清单的数据结构与读写闸门；阶段 C（证据分组与上下文优化）的接口在本说明书中给出定义，其检索与排序算法本身不在本期实现范围内。

明确不在范围内的事项：模型权重与推理缓存的管理；模型选型与提示词设计；跨工作空间的分布式事务；差分隐私发布机制；备份系统与第三方副本的物理清除执行（本设计仅定义墓碑门控与恢复路径的策略检查）；具体向量索引与图数据库的选型（仅预留水位协议）。本说明书遵循研究设计报告的口径：不宣称形成新标准，不将拟议实验写成已完成结果，未在本文中给出任何虚构的性能数字。

2 总体设计

2.1 设计原则

全部实现服从研究设计报告确立的四条原则。第一，历史日志只追加：正常修订产生新记录，不覆盖旧版本，旧快照可重建。第二，效果必经网关：所有外部写操作只能通过执行网关发出，模型与智能体进程可以提出记录建议，但不能自行签发授权或把自己的成功描述写成已验证回执。第三，投影必须带水位：任何由异步过程维护的索引与缓存，都标注 applied_sequence 与 validation_epoch，关键读取与动作准入在投影落后时执行闸门逻辑。第四，职责由代码与数据库共同实现：唯一约束处理请求去重，事务处理预算预留，写入服务处理类型与版本，网关处理权限与连接器契约，读取服务处理策略与水位，不允许只在 JSON schema 中声明一个字段就宣称保证成立。

2.2 模块划分与职责

系统划分为七个模块与一个存储层。模块之间只通过第 5 章定义的操作契约交互，模型、检索器与工作流引擎可替换，但必须通过相同契约接入。

表 3 模块划分与职责

模块	职责	关键机制	研究报告对应
M1 写入服务	接收提案，执行请求去重、schema 校验、类型与引用检查、批内拓扑检查，分配提交序号与 recorded_at，推进验证纪元	串行提交协议、request_id 唯一约束、reducer 版本管理	第 4、7.1 节
M2 执行网关	领取 outbox、重验证前提与授权、记录尝试、调用连接器、持久化回执与观察、结算或保留预留、对账	outbox 模式、效果状态机、幂等契约执行	第 6、7 章，附录 A
M3 读取与投影服务	维护当前版本表与任务状态，执行支持求值、失效标记、上下文投影与检索候选生成	投影水位、epoch 闸门、绑定单元组装	第 5、8 节
M4 策略服务	授权签发校验与撤销、权限检查点、数据标签传播、删除域与墓碑管理	operation_digest 校验、policy_epoch	第 9 节
M5 连接器适配层	封装外部系统，声明能力契约：幂等键范围与期限、回执查询方法、可见性与最终性、条件更新支持	连接器注册表与契约元数据	第 6.2、7.7 节
M6 索引工作者	异步维护全文、向量与图索引，按提交序号增量应用，写水位	applied_sequence 与 epoch 标注	第 12.1 节
M7 审计与回放工具	按提交历史回放状态、导出证据链、执行确定性测试	确定性 reducer、select_b 分支过滤	第 4、13 节
S 存储层	PostgreSQL（记录、账本、outbox）与对象存储（正文、附件、渲染输入）	唯一约束、可串行化事务、不可变对象版本	第 12.1 节

M1 至 M5 在第一版可以部署为同一进程内的五个包，模块边界先以接口契约固定，物理拆分留待阶段 E 按瓶颈证据决定。

2.3 信任边界

系统按可信程度划分三类主体，权限模型以此为基础。第一类是可信计算基：写入服务、执行网关、策略服务与数据库，它们是 recorded_at、提交序号、授权与回执的唯一合法分配者。第二类是半可信主体：外部系统与连接器，其响应一律先落为 Observation，是否采信由主张与评估流程决定；连接器必须显式声明契约能力，

未声明的能力不得被运行时假设。第三类是不可信主体：模型与智能体进程，包括子智能体，它们只能提交提案（proposal），提案经 M1 校验与冲突检查后才成为提交；智能体输出的文本、网页内容或工具结果都不能自行提升权限。

表 4 主体与允许操作

主体	信任级别	允许的操作
写入服务 / 网关 / 策略服务	可信	分配序号与时间戳、签发 Grant、产生 Receipt、推进 epoch
外部连接器	半可信	返回响应（落为 Observation）、按声明契约支持幂等与查询
主智能体进程	不可信	提交 observe、assess、revise、prepare_effect 等提案；读取被授权视图
子智能体	不可信	同上，且授权范围不得超出父授权（第 9.2 节）
人工审批者	可信（经认证通道）	签发与撤销 Grant、裁决冲突主张

2.4 核心抽象与实现的映射

研究设计报告第 4 章的形式化对象到本实现的映射如下，符号含义见研究报告原文。

表 5 形式化对象与实现映射

形式化对象	实现载体	说明
L（提交历史）	commits 表按 (workspace_id, branch_id, sequence) 有序	历史是系统接纳记录的依据，不是外部真值证明
R（记录集合）	records 表 + blob_store	记录不可变，载荷存对象存储，库内存摘要与引用
B（载荷集合）	blob_store 对象（带内容摘要与媒体类型）	大载荷不内联快照，按引用共享
V（投影）	object_heads、task_state、effect_state、search_indexes	全部可由 L 经 reducer 重建
Pi（固定版本转换器）	reducer 函数 + reducer_version 字段	回放只输入已记录的模型输出、回执与审批结果

形式化对象	实现载体	说明
select_b（分支过滤）	记录上的 branch_id 与分支根 sequence 过滤谓词	公共效果与预算日志由工作空间共享，不随分支回滚
as_of / valid_at 双时间查询	读取服务查询参数（sequence 截断 + valid_time 谓词）	两个时间维度分别传参，不允许合并为单时间戳

2.5 技术选型与部署约束

第一版实现落在单个 PostgreSQL 16 及以上实例与一个对象存储（兼容 S3 接口即可）。选用依据：可串行化隔离与谓词锁支持预算账本（第 8.3 节）；SELECT ... FOR UPDATE SKIP LOCKED 支持 outbox 多实例领取；JSONB 存储记录载荷的规范副本；tsvector 满足初期全文检索。向量与图索引是阶段 E 的性能选择，接入时必须遵守第 7.3 节的水位协议。实现语言不限（示例伪代码采用类 Pascal 风格），但 schema 校验、类型检查与权限检查必须由代码执行，不得依赖约定。

部署约束：数据库与网关同机房部署以控制事务与领取延迟；对象存储与数据库之间的写入次序遵循先写对象、后写记录引用；模型调用一律发生在数据库事务之外（第 8.2 节事务边界表）；网关多实例部署时依赖数据库行锁而非进程内状态协调。

3 数据结构详细设计

本章将研究设计报告第 4、5、6、8 章给出的记录草图细化为字段级规格。所有记录共享一个通用信封，载荷按 kind 组成类型联合；记录一经提交即不可变，正常修订以新记录加 supersedes 边表达。载荷的权威副本存于对象存储，库内保存规范化 JSONB 副本以供校验与索引，两者以内容摘要绑定。

3.1 标识、摘要与基本类型

表 6 基本类型定义

类型	表示	规则
RecordId / ObjectId / WorkspaceId / BranchId	UUIDv7 字符串（36 字符）	租户范围内不可猜测；UUIDv7 的时间有序性利于索引局部性；不得由模型生成后直接采信，一律由写入服务分配或校验格式与唯一性
Digest	SHA-256 十六进制，64 字符	用于内容绑定与完整性校验；不对外公开敏感低熵内容的摘要（第 9.5 节）
Timestamp	RFC 3339 字符串，一律 UTC	系统时间（recorded_at）由可信写入服务分配；valid_time 与 event_time 可为区间或未知
Principal	结构体 {kind, id}	kind 取 service、agent、human、connector；id 为稳定身份，禁止用显示名称充当
PolicyDomain	枚举字符串	记录归属的策略域，删除与访问策略按域解析
Span	结构体（按模态取一种）	text:{offset,length}；pdf:{page,char_range}；image:{region_polygon}；av:{start_ms,end_ms}；table:{version,row_keys}
Money / Quantity	十进制字符串 + 显式单位字段	禁止浮点数表示金额与计量；单位为绑定单元组成部分
JsonBlob	对象存储引用 {blob_id, digest, media_type, size}	超过 4 KB 的载荷一律外置；库内 JSONB 仅保留规范化副本

UUIDv7 之外允许 ULID，但同一部署内必须统一，且两种均满足不可猜测性要求时优先 UUIDv7（生态成熟度）。

3.2 记录通用信封 Record

每个记录（无论何种 kind）都携带下列信封字段。字段名与语义与研究设计报告第 4 章一致，此处补充类型与约束。

表 7 Record 通用信封字段

字段	类型	约束	说明
record_id	RecordId	全局唯一，由写入服务分配	标识一个确定版本
object_id	ObjectId	非空	逻辑对象标识，跨版本稳定；实体合并后旧 object_id 仍保留
workspace_id	WorkspaceId	非空	归属工作空间
branch_id	BranchId	非空，默认 main	归属分支；分支记录对其他分支不可见
kind	枚举	见附录 A 注册表	记录类型的判别字段
schema_version	整数	大于等于 1	该 kind 载荷的 schema 版本；演进规则见 3.15 节
authenticated_writer	Principal	由认证通道确定	已认证的实际写入者，不采信载荷内自报作者
asserted_source	Principal 或 null	载荷声明	材料声称来自谁，与 authenticated_writer 分开保存
recorded_at	Timestamp	由写入服务分配	系统何时知道；与 valid_time、event_time 分离（3.9 节）
payload_ref	JsonBlob	digest 必须与库内副本一致	载荷权威副本引用
prior_revision_ids	RecordId 数组	可空	本记录修订了哪些旧版本，构成修订链
typed_refs	引用数组	见 3.13 节	每项 {role, layer, target_record_id, target_object_id, note}
policy_domain	PolicyDomain	非空	删除与访问策略解析依据
commit_id	CommitId	由写入服务回填	所属提交，入 commit 后不可变

3.3 提交 Commit 与快照 Snapshot

表 8 Commit 字段

字段	类型	约束	说明
commit_id	CommitId	全局唯一	提交标识
workspace_id / branch_id	ID	联合唯一序号	与 sequence 构成 (workspace, branch, sequence) 唯一键
sequence	整数	分支内单调递增、无空洞	由写入服务在串行协议下分配（第 8.1 节）
previous_commit_id	CommitId	必须等于提交时分支头	冲突时不静默合并，返回 E201 要求重检查
record_ids	RecordId 数组	非空，批内拓扑合法	批内允许引用同批前序记录，但不得引用未来记录（不变量 I2）
reducer_version	整数	非空	本提交引入的 reducer 版本；回放按版本切换转换逻辑
request_id	字符串	(workspace, request_id) 唯一	幂等键；重复且摘要相同返回原结果，不同则拒绝（I3）
request_digest	Digest	非空	请求内容摘要，配合 request_id 防止一键两用
committed_at	Timestamp	写入服务分配	提交时间

表 9 Snapshot 字段

字段	类型	说明
workspace_id / branch_id	ID	快照归属
branch_head	{sequence, commit_id}	分支头位置
reducer_version	整数	重建投影所需的 reducer 版本链
validation_epoch	整数	失效传播纪元；读取闸门比较对象
policy_epoch	整数	访问策略纪元；删除与权限变更推进

快照不是数据的另一份拷贝，而是定位符：恢复或分支时按 branch_head 截断提交历史，经 reducer 重建投影。大载荷与不可变记录在分支与快照间共享引用，不复制。

3.4 证据类记录 Observation

Observation 只断言某来源在某时刻返回了某段内容，不直接升级为真值。多模态证据以 Span 精确定位；机器转录与 OCR 不是对原始观察的改写，而是一条新的派生记录，通过 extracted_from 边指回源观察。

表 10 Observation 载荷字段

字段	类型	约束	说明
source	结构体	非空	{connector_identity, fetched_via, url_or_handle, content_version}；content_version 为 ETag、版本号或内容摘要
modality	枚举	text/pdf/image/audio/video/table/api_response	证据模态，决定 span 结构
span	Span	与 modality 匹配	原始内容的精确位置；引用必须落到具体片段，不得只给整篇
acquired_at	Timestamp	事件时间语义	采集时刻（被讨论事件何时发生维度之一）
acquisition_method	结构体	非空	{tool, tool_version, parameters_digest}，采集方式本身可追溯
content	JsonBlob 或内联短文本	摘要必填	原始内容或其片段；超过阈值外置对象存储
media_type	字符串	MIME 类型	内容类型
derived_from	RecordId 或 null	指向另一 Observation	转录、OCR、格式转换等派生来源；原始采集为 null
declared_freshness	结构体	可选	来源自述的时效信息，仅作参考，不等于系统的 freshness 状态

代码 1 Observation 载荷示例

```
// Observation 载荷示例（PDF 引用页与片段）
{
  "source": { "connector_identity": "web.fetch@v2", "url_or_handle":
"https://example.org/ds-spec",
              "content_version": "etag:9f2c1a" },
  "modality": "pdf",
  "span": { "page": 12, "char_range": [880, 1264] },
  "acquired_at": "2026-10-20T09:31:07Z",
```

```
  "acquisition_method": { "tool": "pdf.extractor", "tool_version": "1.7",
"parameters_digest": "sha256:..." },
  "content": { "blob_id": "b01", "digest": "sha256:...", "media_type": "application/pdf",
"size": 182044 },
  "derived_from": null
}
```

3.5 主张类记录 Claim

Claim 保存结构化命题或受约束的自然语言文本，并显式携带适用范围、单位与时间限定。相同姓名不得自动合并为同一实体；实体合并是独立的、可撤销的记录（kind 为 ENTITY_MERGE，见附录 A）。

表 11 Claim 载荷字段

字段	类型	约束	说明
proposition	结构化三元组或 constrained_text	二选一	结构化时为 {subject_entity, predicate, object, qualifiers}；文本时必须携带约束标签
entity_ids	ObjectId 数组	可为空	命题涉及的实体；引用实体注册记录而非名称字符串
scope	结构体	非空	适用范围：数据集、项目、租户等边界条件
units	字符串或 null	数量命题必填	计量单位，属于绑定单元，投影时不可剥离
valid_time	时间结构	可为区间或未知	命题何时适用（负责人自周一起变更的周一维度）
event_time	时间结构	可选	被讨论事件何时发生（活动计划周五举行的周五维度）
proposer	Principal	非空	提出者；与 authenticated_writer 可以不同
candidate_status	枚举	candidate 或 asserted	初始为候选；是否采信由评估与支持关系决定，不在此字段表达
negation_conditions	数组	可空	命题不成立的边界条件，属于绑定单元（第 7.4 节）

3.6 支持关系记录 Justification

普通 supports 列表无法区分必须同时依赖与独立充分两种情形，Justification 保存 AND 组与 OR 分支组成的支持电路：组内证据为合取（缺一即失效），组间为析取（任一组合格即存在支持）。支持与反证分别建组；推理规则与验证器本身也是记录，被规则边引用并携带版本。

表 12 Justification 载荷字段

字段	类型	约束	说明
target_claim_id	RecordId	指向 Claim	被论证的主张
support_groups	数组（OR of AND）	每组为 RecordId 数组	每个 AND 组内证据全部合格时该组合格；任一组合格即存在合格支持
refute_groups	数组（OR of AND）	同上	反证分支，与支持分支独立求值
rule_refs	RecordId 数组	可空	推理规则或验证器记录及其版本；规则失效等同于全部依赖其的组失效
assumption_refs	RecordId 数组	可空	显式假设；循环主张不能凭互引用产生支持，必须有可用观察或显式假设作根
justification_note	字符串	可空	论证说明

代码 2 Justification 支持电路示例

```
// 支持电路示例：C1 依赖 (E1 AND E2) OR E3；反证分支 E4
Justification {
  target_claim_id: "rec_claim_c1",
  support_groups: [ ["rec_obs_e1", "rec_obs_e2"],   // AND 组 1
                    ["rec_obs_e3"] ],                // AND 组 2（独立充分）
  refute_groups:  [ ["rec_obs_e4"] ],
  rule_refs:      ["rec_rule_validator_v3"],
  assumption_refs: []
}
// E1 被撤回时组 1 失效；E3 仍合格则 C1 仍有支持（研究报告 5.2 节语义）
```

求值语义：Eligible 仅表示在指定查询范围、as_of 序号与策略下，证据可被使用且相关验证尚有效，不表示其为真。四态结果（支持位、反证位）由第 7.2 节算法计算；数据结构只维护已声明的依赖，未记录的隐含前提造成的漏失效不属于结构承诺。

3.7 评估记录 Assessment

表 13 Assessment 载荷字段

字段	类型	约束	说明
target_record_id	RecordId	指向 Claim 或 Justification	被评估对象（固定版本）
method	字符串	非空	评估方法标识（人工核验、模型评估、程序校验器）
evaluator	结构体	非空	{principal, model_id?, model_version?}；模型评估必须携带模型标识与版本
calibration_ref	RecordId 或 null	可选	校准集版本引用；统计量必须可追溯其校准依据
verdict	枚举	verified / contradicted / uncertain / withdrawn	评估结论，作用于目标版本
confidence_stat	结构体	可选	模型置信度为附加统计量，不得作为准入唯一依据（研究报告 5.3 节）
valid_window	时间结构	可选	评估结论的有效期

3.8 工作类记录 TaskRevision

任务以修订序列表达：每次目标、范围或验收条件的变化都产生新的 TaskRevision，通过 supersedes 边链接前一版本，任务循环不伪装成无环计划。任务的当前工作状态（ACTIVE、PENDING_REEVALUATION 等）是投影，存于 task_state 表，不写入记录。

表 14 TaskRevision 载荷字段

字段	类型	约束	说明
task_object_id	ObjectId	跨修订稳定	任务逻辑对象
goal	字符串	非空	目标描述
scope	结构体	非空	任务边界与资源边界
acceptance_refs	RecordId 数组	非空	验收谓词记录；任务完成必须满足其声明（第 6.5 节状态机）

字段	类型	约束	说明
depends_on_task_refs	RecordId 数组	可空	前置任务及其版本；失效传播命中前置时任务进入待重评
artifact_requirements	数组	可空	产物要求：类型、格式、验收摘要
iteration	整数	非负	迭代序号；同一任务的多次循环以新修订表达
supersedes	RecordId 或 null	前一修订	修订链；基线不匹配时 revise 操作被拒绝（E203）

3.9 动作意图 ActionIntent

字段与研究设计报告第 6 章一致。意图描述一次具体外部操作；同一任务可有多个意图，每个意图可有多次受控尝试。意图 ID、尝试 ID、工具请求 ID 与外部作业 ID 分开保存，任何合并都会破坏对账语义。

表 15 ActionIntent 载荷字段

字段	类型	约束	说明
intent_id	ID	工作空间内唯一	逻辑意图标识，重试沿用，不随尝试变化
task_revision_id	RecordId	指向具体任务修订	意图服务于哪个任务版本
operation	字符串	连接器注册的操作名	如 compute.job.submit、email.send
connector_identity / connector_version	字符串	非空	目标连接器身份与版本，参与 operation_digest
executor_principal	Principal	非空	执行主体；名称须在授权前解析为稳定身份，执行时不得悄悄重指向（第 9.2 节）
target_ids	ID 数组	非空	真实目标对象标识，参与 operation_digest
arguments_ref	JsonBlob	摘要必填	冻结后的参数，规范化摘要参与 operation_digest
artifact_revision_ids	RecordId 数组	可空	操作涉及的确切产物内容版本
precondition_refs	RecordId 数组	可空	前置条件：证据版本、任务修订、关键主张；dispatch 前重验证
context_manifest_id	RecordId	高影响动作必填	决策时实际看到的上下文清单

字段	类型	约束	说明
idempotency_contract	结构体	见 6.2 节	{mode, key, key_scope, key_expiry, retry_policy}
resource_reservation_refs	ID 数组	可空	原子预留引用（第 7.5 节）

3.10 授权记录 AuthorizationGrant

授权是运行时强制执行的控制数据，不是描述性文本。Grant 只能由被允许的签发者（人工审批者或具备委派权限的服务）创建；子授权的范围不得超出父授权。批准 v7 不自动授权 v8，但仅仅出现 v8 也不使冻结为 v7 的动作失效——需要重检查的是具体策略与前置条件，由网关在 dispatch 时执行（第 7.6 节步骤 3）。

表 16 AuthorizationGrant 载荷字段

字段	类型	约束	说明
grant_id	ID	全局唯一	授权标识
issuer / grantee	Principal	签发者须具备权限	签发者与被授权主体
parent_grant_id	ID 或 null	委派链	非空时范围必须收敛于父授权（第 9.2 节检查）
operation_digest	Digest	非空	规范化操作摘要，覆盖字段见 9.1 节；网关按摘要匹配而非按名称
scope	结构体	非空	目标范围、参数约束、数据标签上限
purpose	字符串	非空	用途绑定；超出用途的使用被拒绝
expiry	Timestamp	非空	过期时刻；网关重验证包含此项
revocation_handle	ID	非空	撤销句柄；撤销立即影响后续 dispatch，不能收回已发出的外部请求
use_limit / uses_consumed	整数	use_limit 大于等于 1	按逻辑意图计量；尝试重试不额外获得授权（第 7.5 节）
allowed_destinations	数组	非空	允许的目标清单；读取权限与向某目标披露数据的权限分开声明
policy_version	整数	非空	签发时的策略版本，审计用

3.11 尝试 Attempt、回执 Receipt 与产物 Artifact

表 17 Attempt 载荷字段

字段	类型	约束	说明
attempt_id	ID	全局唯一	一次受控尝试的标识
intent_id	ID	非空	归属逻辑意图
gateway_principal	Principal	仅网关可写	执行网关实例
started_at	Timestamp	网关分配	外部调用开始前的落库时刻（先记录后调用）
dispatch_digest	Digest	非空	本次实际发出请求的规范化摘要，与 intent 冻结摘要核对
idempotency_key_state	结构体	与契约匹配	发出的键、范围、有效期；键过期后重试不得声称安全（E603）
external_job_id	字符串或 null	远端返回	外部作业标识，仅当连接器契约提供
attempt_status	枚举	第 6.3 节	STARTED / RESPONSE_PERSISTED / NO_RESPONSE

表 18 Receipt 载荷字段

字段	类型	约束	说明
attempt_id	ID	非空	回执先属于具体尝试，再按意图聚合
outcome	枚举	第 6.1 节	CONFIRMED_SUCCEEDED / CONFIRMED_NOT_APPLIED / OUTCOME_UNKNOWN / PARTIAL
classification_basis	结构体	非空	连接器契约条款引用：为何该响应对应该分类；无契约支撑的成功描述不得分类为确认
evidence_observation_id	RecordId	非空	原始响应先落为 Observation，回执引用之
partial_items	数组	仅 PARTIAL	复合操作拆分后的逐项结果，供逐项对账
recorded_by	Principal	仅网关或可信接入	智能体不得自行签发回执

表 19 Artifact 载荷字段

字段	类型	约束	说明
artifact_object_id	ObjectId	跨版本稳定	产物逻辑对象
content_version	整数	非空	内容版本；操作摘要绑定确切产物内容
content	JsonBlob	摘要必填	产物正文引用
produced_by_attempt	ID	非空	由哪次尝试产生
data_labels	数组	继承并只增不减	沿输入传播的数据标签（第 9.3 节）

3.12 上下文清单 ContextManifest

上下文不是另一份可随意改写的真相，而是一次有契约的读取结果。ContextManifest 记录模型实际看到了什么：选中的记录与片段、顺序、变换或摘要版本、预算、检索器与策略版本。模型看见了什么必须落实到实际渲染输入（rendered_input_digest），不能只记录一条检索查询。

表 20 ContextManifest 载荷字段

字段	类型	约束	说明
context_id	ID	全局唯一	清单标识
snapshot	Snapshot 定位	非空	读取时的分支头、reducer 版本、两个 epoch
task_revision_id	RecordId	可空	服务于哪个任务版本
selected_items	数组	非空	{record_id, span, transform_id}：选中记录、精确片段与变换版本
rendered_input_ref / rendered_input_digest	JsonBlob / Digest	渲染后必填	实际发送给模型的完整输入及其摘要
required_items	RecordId 数组	可空	标记为必需的绑定单元；被省略时进入 omitted_items_summary
omitted_items_summary	结构体	预算不足时非空	被裁剪内容及原因；必需约束超出预算时必须分阶段、提额或暂停动作，不得静默丢弃（E701）
token_budget / tokenizer_version	整数 / 字符串	非空	预算与分词器版本

字段	类型	约束	说明
retrieval_version	字符串	非空	检索器与排序器版本
policy_epoch	整数	非空	读取时策略纪元；渲染缓存据此失效
inherited_data_labels	数组	非空	所有选中条目标签的并集，随输出传播
expiry	Timestamp	非空	清单有效期，超过后不得作为动作的 context_manifest_id

3.13 关系边类型系统 typed_refs

三种关系不能混为一张无类型图：版本与已声明派生构成 DAG，语义关系允许成环，控制与执行关系按各自状态机验证。每条边携带来源与类型，控制边不得由相似度推导。下表为第一版注册的关系角色；扩展角色必须先登记结构限制，再允许写入。

表 21 关系角色注册表

层 layer	角色 role	语义	结构限制	允许的源与目标 kind
DERIVATION	supersedes	新版本替代旧版本	DAG；指向已存在或同批前序记录	任意 kind 到同 object_id 的旧版本
DERIVATION	derived_from / extracted_from / transcribed_from	派生与转录来源	DAG；不得指向未来记录（I2）	Observation、Claim、Artifact 到 Observation 等
DERIVATION	uses_version	声明使用了哪个确定版本	DAG	TaskRevision、ActionIntent、Justification 到任意记录
SEMANTIC	related_to / similar_to	领域关联与相似	允许成环；不产生支持，不触发失效	Claim 到 Claim、实体到实体
SEMANTIC	same_entity_candidate	疑似同一实体	允许成环；合并须经 ENTITY_MERGE 记录确认	实体记录之间
CONTROL	supports / refutes	支持与反证分组（经 Justification）	按 Justification 结构验证	Justification 到 Observation、Claim

层 layer	角色 role	语义	结构限制	允许的源与目标 kind
CONTROL	depends_on / precondition_of	任务与动作前提	命中失效传播时触发重评（第 7.3 节）	TaskRevision、ActionIntent 到任意记录
CONTROL	authorized_by	授权绑定	摘要匹配校验（第 9.1 节）	ActionIntent 到 Grant
CONTROL	attempt_of / receipt_of	尝试与回执归属	仅网关写入	Attempt 到 ActionIntent；Receipt 到 Attempt
CONTROL	selected_into	上下文选中	指向 ContextManifest	任意记录到 ContextManifest
CONTROL	produced	尝试产物	版本与摘要一致	Attempt 到 Artifact

派生边表示系统记录的使用关系，不是现实世界的因果效应。A 信任 B 与 B 信任 A 可同时存在；但新版本不能把未来版本当作已使用的输入。反向依赖索引（record_refs 反向查询）是失效传播的基础，必须与正向边同事务写入。

3.14 状态与枚举字典

表 22 证据与内容状态枚举

枚举	取值	语义与规则
EvidenceFourState	NONE / SUPPORTED / REFUTED / CONTESTED	由支持位与反证位组合：1/1 为 CONTESTED，必须展示或裁决；不表示形式真值或概率
Freshness	fresh / stale	来源或命题相对过期策略的状态；stale 不等于错误
Visibility	available / restricted / hidden	按主体权限解析；对无权主体连证据存在性也按策略隐藏
Retention	retained / purged	purged 表示正文已进入删除域处理；元数据按策略决定去留
AssessmentStatus	unassessed / verified / contradicted / withdrawn	评估结论投影；撤回支持不是新增反证

表 23 执行与资源状态枚举

枚举	取值	语义与规则
EffectState	PROPOSED / PREPARED / IN_FLIGHT / OUTCOME_UNKNOWN / CONFIRMED_SUCCEEDED / CONFIRMED_NOT_APPLIED / CANCELLED_BEFORE_DISPATCH / PARTIAL	第 6.1 节状态机；UNKNOWN 期间保留资源与授权占用
AttemptStatus	STARTED / RESPONSE_PERSISTED / NO_RESPONSE	NO_RESPONSE 在超时后仍为尝试级状态，不直接判定意图失败
ReservationState	reserved / uncertain / committed / released	uncertain 对应意图 OUTCOME_UNKNOWN；released 须满足释放守卫（第 6.4 节）
GrantStatus	active / expired / revoked / exhausted	过期与撤销由网关重验证判定；次数用尽为 exhausted
IdempotencyMode	KEYED / QUERY_ONLY / NONE	KEYED：远端支持键；QUERY_ONLY：可查询对账；NONE：默认暂停自动重发（E604）
TaskStatus	DRAFT / ACTIVE / PENDING_REEVALUATION / COMPLETED / CANCELLED	COMPLETED 的守卫为验收谓词全部通过；PENDING_REEVALUATION 由失效传播触发

3.15 schema 演进规则

schema_version 只增不减。新增可选字段递增次版本；改变字段语义、删除字段或收紧约束必须递增主版本并同步升级 reducer，旧版本载荷在回放时按其声明版本的解释器处理。写入服务按注册表（附录 A）校验当前接受的最低与最高版本，拒绝无法解释的版本（E102）。载荷中不允许出现注册表之外的字段名，防止以扩展名义夹带未声明语义。

4 数据库物理设计

第一版实现落在单个 PostgreSQL 实例：记录、事务、JSONB 载荷副本与必要索引全部入库，正文与附件放对象存储，初期全文检索直接使用数据库 tsvector。研究设计报告第 12.1 节的存储对象清单在本章落为具体表结构；关系数据库、图数据库与向量索引是后续阶段的性能选择，不是语义依赖。

4.1 表清单与一致性要求

表 24 表清单

表	职责	一致性要求	对应模块
workspace / branch	工作空间、分支与两个纪元的当前值	强一致；epoch 推进与失效事务同库	M1
commit	提交序列（历史日志 L）	与关键状态变更同事务；序号无空洞	M1
record	不可变记录（记录集合 R）	插入后禁 UPDATE/DELETE（触发器强制）	M1
record_ref	类型化正反向引用边	与 record 同事务写入；失效索引读此表	M1/M3
object_heads	逻辑对象当前版本	强一致；用于准入与读取	M3
task_state	任务当前工作状态	强一致	M3
effect_state	意图效果状态	强一致；与 outbox、预留同事务	M2
grant / grant_usage	授权与按意图的次数占用	强一致；原子检查	M4
budget_ledger / reservation	预算账本与资源预留	强一致；可串行化事务内原子预留	M1/M2
effect_outbox / attempt / receipt_view	待发送动作、尝试与回执投影	与准备动作同事务写入 outbox	M2
invalidation_queue / index_watermark	失效队列与异步索引水位	队列强一致入队；水位由工作者推进	M3/M6
blob_object / tombstone	对象存储登记与删除墓碑	墓碑先行持久化	M4/S

4.2 核心表 DDL

下列 DDL 为第一版权威定义，约束即不变量的第一道实现（映射关系见 8.5 节）。

代码 3 commit 与 record 表

```
-- 提交与记录（历史日志 L 与记录集合 R）
CREATE TABLE commit (
  commit_id        uuid PRIMARY KEY,
  workspace_id     uuid NOT NULL,
  branch_id        uuid NOT NULL DEFAULT 'main',
```

```
  sequence         bigint NOT NULL,
  previous_commit_id uuid NOT NULL,
  reducer_version  int  NOT NULL,
  record_ids       uuid[] NOT NULL,
  request_id       text NOT NULL,
  request_digest   text NOT NULL,
  committed_at     timestamptz NOT NULL,
  UNIQUE (workspace_id, branch_id, sequence),
  UNIQUE (workspace_id, request_id)          -- I3：幂等键唯一
);

CREATE TABLE record (
  record_id     uuid PRIMARY KEY,
  object_id     uuid NOT NULL,
  workspace_id  uuid NOT NULL,
  branch_id     uuid NOT NULL DEFAULT 'main',
  kind          text NOT NULL,
  schema_version int NOT NULL CHECK (schema_version >= 1),
  authenticated_writer jsonb NOT NULL,
  asserted_source jsonb,
  recorded_at   timestamptz NOT NULL,
  payload_digest text NOT NULL,
  payload_ref   jsonb NOT NULL,
  payload       jsonb NOT NULL,   -- 规范化副本，仅供校验与索引
  policy_domain text NOT NULL,
  commit_id     uuid NOT NULL REFERENCES commit(commit_id),
  sequence      bigint NOT NULL   -- 冗余分支序号，供 as_of 截断
);
-- 不可变性由触发器强制：禁止 UPDATE / DELETE（回放与测试依赖）
CREATE TRIGGER record_immutable BEFORE UPDATE OR DELETE ON record
  FOR EACH ROW EXECUTE FUNCTION reject_mutation();
CREATE INDEX ON record (workspace_id, branch_id, sequence);
CREATE INDEX ON record (object_id);
CREATE INDEX ON record (kind);
```

代码 4 record_ref 与投影表

```
-- 类型化引用边（正反向查询共用一张表）
CREATE TABLE record_ref (
  src_record_id uuid NOT NULL REFERENCES record(record_id),
  ref_role      text NOT NULL,   -- 3.13 节注册表
  ref_layer     text NOT NULL CHECK (ref_layer IN ('DERIVATION','SEMANTIC','CONTROL')),
  dst_record_id uuid NOT NULL REFERENCES record(record_id),
  dst_object_id uuid NOT NULL,
  note          text,
  PRIMARY KEY (src_record_id, ref_role, dst_record_id)
);
CREATE INDEX ON record_ref (dst_record_id);          -- 反向依赖：失效传播入口
CREATE INDEX ON record_ref (dst_record_id, ref_layer);

-- 当前版本与任务状态（投影 V 的强一致部分）
CREATE TABLE object_heads (
  workspace_id uuid NOT NULL,
  object_id    uuid NOT NULL,
  head_record_id uuid NOT NULL REFERENCES record(record_id),
```

```
  head_sequence bigint NOT NULL,
  updated_at   timestamptz NOT NULL,
  PRIMARY KEY (workspace_id, object_id)
);

CREATE TABLE task_state (
  workspace_id uuid NOT NULL,
  task_object_id uuid NOT NULL,
  status       text NOT NULL CHECK (status IN
    ('DRAFT','ACTIVE','PENDING_REEVALUATION','COMPLETED','CANCELLED')),
  current_revision uuid NOT NULL REFERENCES record(record_id),
  acceptance_status jsonb NOT NULL,  -- 各验收谓词的当前判定
  reevaluation_reason text,
  PRIMARY KEY (workspace_id, task_object_id)
);
```

代码 5 授权、预算与预留表

```
-- 授权与使用账目
CREATE TABLE grant_record (
  grant_id     uuid PRIMARY KEY,
  issuer       jsonb NOT NULL,
  grantee      jsonb NOT NULL,
  parent_grant_id uuid REFERENCES grant_record(grant_id),
  operation_digest text NOT NULL,
  scope        jsonb NOT NULL,
  purpose      text NOT NULL,
  expiry       timestamptz NOT NULL,
  revocation_handle uuid NOT NULL,
  use_limit    int NOT NULL CHECK (use_limit >= 1),
  uses_consumed int NOT NULL DEFAULT 0,
  allowed_destinations jsonb NOT NULL,
  policy_version int NOT NULL,
  status       text NOT NULL DEFAULT 'active'
);
CREATE TABLE grant_usage (
  grant_id  uuid NOT NULL REFERENCES grant_record(grant_id),
  intent_id uuid NOT NULL,
  taken_at  timestamptz NOT NULL,
  PRIMARY KEY (grant_id, intent_id)   -- 同一意图重试不重复计数
);

-- 预算账本与资源预留（I5：预留加已确认消耗不超额度）
CREATE TABLE budget_ledger (
  resource_key text PRIMARY KEY,      -- 如 ws:compute.gpu.a100
  limit_amount numeric NOT NULL CHECK (limit_amount >= 0),
  reserved_amount numeric NOT NULL DEFAULT 0,
  committed_amount numeric NOT NULL DEFAULT 0,
  CHECK (reserved_amount + committed_amount <= limit_amount)
);
CREATE TABLE reservation (
  reservation_id uuid PRIMARY KEY,
  intent_id      uuid NOT NULL,
  resource_key   text NOT NULL REFERENCES budget_ledger(resource_key),
  amount         numeric NOT NULL CHECK (amount > 0),
```

```
  state          text NOT NULL CHECK (state IN
    ('reserved','uncertain','committed','released')),
  created_at     timestamptz NOT NULL,
  settled_at     timestamptz
);
CREATE INDEX ON reservation (intent_id);
```

代码 6 效果执行表

```
-- 效果状态、outbox 与尝试
CREATE TABLE effect_state (
  workspace_id uuid NOT NULL,
  intent_id    uuid NOT NULL,
  state        text NOT NULL CHECK (state IN ('PROPOSED','PREPARED','IN_FLIGHT',
    'OUTCOME_UNKNOWN','CONFIRMED_SUCCEEDED','CONFIRMED_NOT_APPLIED',
    'CANCELLED_BEFORE_DISPATCH','PARTIAL')),
  unresolved_reason text,
  updated_epoch bigint NOT NULL,
  PRIMARY KEY (workspace_id, intent_id)
);

CREATE TABLE effect_outbox (
  outbox_id   uuid PRIMARY KEY,
  intent_id   uuid NOT NULL,
  state       text NOT NULL DEFAULT 'PENDING' CHECK (state IN
    ('PENDING','CLAIMED','DONE','ABANDONED')),
  claim_token uuid,
  claim_expires_at timestamptz,
  attempts_count int NOT NULL DEFAULT 0,
  next_retry_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (intent_id)
);

CREATE TABLE attempt (
  attempt_id  uuid PRIMARY KEY,
  intent_id   uuid NOT NULL,
  gateway_principal jsonb NOT NULL,
  started_at  timestamptz NOT NULL,
  dispatch_digest text NOT NULL,
  idempotency_key_state jsonb,
  external_job_id text,
  attempt_status text NOT NULL CHECK (attempt_status IN
    ('STARTED','RESPONSE_PERSISTED','NO_RESPONSE'))
);
CREATE INDEX ON attempt (intent_id);
```

代码 7 失效、水位与删除域表

```
-- 失效传播、水位与删除域
CREATE TABLE workspace_epoch (
  workspace_id   uuid PRIMARY KEY,
  validation_epoch bigint NOT NULL DEFAULT 1,
  policy_epoch     bigint NOT NULL DEFAULT 1,
  head_sequence    bigint NOT NULL DEFAULT 0
);
```

```
CREATE TABLE invalidation_queue (
  item_id     bigserial PRIMARY KEY,
  workspace_id uuid NOT NULL,
  epoch       bigint NOT NULL,        -- 触发本次失效的验证纪元
  root_record_id uuid NOT NULL,       -- 失效根（被撤回/更正/删除的记录）
  status      text NOT NULL DEFAULT 'PENDING' CHECK (status IN
    ('PENDING','PROCESSING','DONE')),
  created_at  timestamptz NOT NULL
);

CREATE TABLE invalidation_mark (
  workspace_id uuid NOT NULL,
  record_id    uuid NOT NULL,         -- 受影响的下游记录
  path_role    text NOT NULL,         -- 命中的依赖路径类型
  via_record_id uuid NOT NULL,        -- 传播路径上的直接依赖者
  epoch        bigint NOT NULL,
  PRIMARY KEY (workspace_id, record_id, via_record_id)
);

CREATE TABLE index_watermark (
  index_name      text PRIMARY KEY,
  applied_sequence bigint NOT NULL,
  validation_epoch bigint NOT NULL,
  updated_at      timestamptz NOT NULL
);

CREATE TABLE tombstone (
  object_id    uuid NOT NULL,
  record_id    uuid,                  -- 记录级或对象级删除
  payload_digest text,                -- 载荷级删除
  policy_domain text NOT NULL,
  purged_at    timestamptz NOT NULL,
  policy_epoch bigint NOT NULL,       -- 先于清理持久化（第 7.8 节）
  PRIMARY KEY (object_id, record_id, payload_digest)
);

CREATE TABLE blob_object (
  blob_id     uuid PRIMARY KEY,
  digest      text NOT NULL,
  media_type  text NOT NULL,
  size_bytes  bigint NOT NULL,
  storage_uri text NOT NULL,
  deletion_state text NOT NULL DEFAULT 'live' CHECK (deletion_state IN
    ('live','pending_purge','purged')),
  data_labels jsonb NOT NULL DEFAULT '[]'
);
```

4.3 索引设计

除 DDL 中内联的索引外，读取路径补充以下索引：record (kind, recorded_at) 支持按类型的近期扫描；record (payload->>'target_claim_id') 支持主张到论证的反查（JSONB GIN 索引按查询模式在阶段 A 压测后决定取舍）；record_ref (src_record_id, ref_layer) 支持正向遍历；attempt (started_at) 支持超时巡检；effect_outbox (state, next_retry_at) 部分索引 WHERE state IN ('PENDING','CLAIMED') 支撑网关领取扫描。所有索引的选择依据与复杂度注明条件见第 11 章，不得以加索引替代闸门逻辑。

4.4 约束与不变量的第一道防线

数据库层实现的不变量：I3 由 commit 表两个唯一约束实现，request_id 重复但 request_digest 不同时，写入服务在捕获唯一冲突后比对摘要并拒绝（E202）；I5 由 budget_ledger 的 CHECK 约束加可串行化事务内的条件更新实现，CHECK 只是最后防线，正常路径是先 SELECT FOR UPDATE 后判断余额；I1 由 record_immutable 触发器实现；I4 由 object_heads 的条件更新（WHERE head_sequence < 新序号）实现。I2、I6 至 I10 无法用声明式约束表达，由写入服务与网关在事务内执行程序化检查检查点见第 5、7 章。

4.5 增长控制与归档

第一版不分区。阶段 E 提供两个预留：record 与 commit 按月分区的方案（以 sequence 范围切分，配合回放按序号截断）；历史压缩方案（将已无活跃依赖且超过审计保留期的旧提交载荷迁移冷存储，仅保留摘要与拓扑）。压缩只能改变存储形式，不得丢弃仍需审计的版本、未完成动作或删除域尚未处理完成的记录。对象存储按 blob 的 deletion_state 生命周期规则清理，清理以墓碑存在为前提。

5 操作接口详细设计

本章细化研究设计报告第 10.1 节的九个关键操作契约。接口以进程内函数签名描述（传输层可为进程调用、HTTP 或消息，契约不变）；每个操作给出签名、前置条件、处理步骤、事务边界与错误码。错误码目录见第 10 章。

5.0 通用约定

所有写操作的请求信封携带：workspace_id、branch_id、expected_head（或明确的读版本集合）、request_id、request_digest、principal 凭证。写入服务对 (workspace_id, request_id) 去重：重复请求若摘要一致，直接返回原提交结果；不一致则拒绝（E202），不允许一个幂等键用于两个请求。

所有写操作遵循统一骨架：校验身份与权限，校验 schema 与引用，检查提交基线（expected_head），在一个数据库事务内完成记录写入、账本变更与失效队列入队，提交后返回新快照定位。模型调用、外部工具调用一律不在事务内。读取操作显式接收 as_of（sequence 截断）与 valid_at 参数，两个时间维度不得合并。

5.1 observe：记录观察

代码 8 observe 签名

```
observe(ws, branch, expected_head, request, principal,
        source, modality, span, content_blob, acquired_at,
        acquisition_method) -> {record_id, commit_id, snapshot}
```

前置条件：来源身份与内容位置可记录（source 结构完整、span 与 modality 匹配）；content 已写入对象存储且摘要一致；principal 具备该采集通道的写入权限。

1 解析并规范化 source 与 span，按 3.4 节校验载荷；派生内容（转录、OCR）必须携带 derived_from 指回源观察。

2 创建 Observation 记录（candidate 语义：不直接升级为真值）。

3 若 span 命中已删除域（对账墓碑），拒绝写入（E802），避免删除内容经观察通道复活。

4 提交并触发索引工作者登记新证据（水位推进由 M6 异步完成）。

事务边界：单事务（记录 + commit + 引用边 + blob_object 登记）。失败语义：对象存储已写而事务失败时，孤儿 blob 由清理任务按无引用回收。错误码：E101 schema 不符、E202 幂等冲突、E802 命中删除域。

5.2 assess：主张评估

代码 9 assess 签名

```
assess(ws, branch, expected_head, request, principal,
       claim: {proposition, entities, scope, units, valid_time},
       justification?: {support_groups, refute_groups, rule_refs},
```

```
        assessment?: {method, evaluator, verdict, calibration_ref})
    -> {claim_id, justification_id?, assessment_id?, commit_id}
```

前置条件：引用的每个证据版本存在且 principal 有权读取；引用的规则记录版本存在；评估者对目标域有权限。

1 校验 Claim 载荷；实体引用必须指向实体注册记录，相同名称不触发自动合并。

2 校验 Justification：每个证据引用存在；规则引用存在；不检查证据真值（真值不是结构承诺）。

3 登记 CONTROL 层 supports / refutes 边，与记录同事务写入 record_ref。

4 若携带 Assessment，校验 evaluator 标识完整（模型评估必须带模型与版本）。

5 触发支持电路增量重算（第 7.2 节），目标主张进入待重算集合。

事务边界：单事务。并发语义：多个 Agent 同时对同一主张建立 Justification 属正常并发，支持电路为可合并结构（OR 分支追加），无需互斥。错误码：E301 引用版本不存在、E302 引用未来记录、E303 无读取权限。

5.3 revise：版本修订

代码 10 revise 签名

```
revise(ws, branch, expected_head, request, principal,
       object_id, prior_revision_ids, new_payload) -> {record_id, commit_id}
```

前置条件：基线版本匹配——prior_revision_ids 必须等于该对象当前 head 的全部前序（乐观并发检查）；新载荷通过 schema 校验；typed_refs 拓扑合法。

1 检查 object_heads：head_record_id 的修订链与 prior_revision_ids 一致，否则返回 E203 并附当前 head，调用方重新检查后重提，不静默最后写入获胜。

2 写入新记录，登记 supersedes 边（DERIVATION，DAG）。

3 更新 object_heads（条件更新保证 I4）。

4 若修订使旧版本的某些主张不再被引用为前提，不做删除：旧版本仍可按 as_of 查询（当时知道什么）。

事务边界：单事务（记录 + heads + 边）。错误码：E102 schema 版本不可解释、E203 基线不匹配、E301 引用不存在。

5.4 invalidate：失效传播

代码 11 invalidate 签名

```
invalidate(ws, request, principal,
           target: {record_id | object_id | payload_digest},
           reason: {withdrawn | corrected | superseded | purged})
    -> {validation_epoch, queue_item_ids}
```

前置条件：操作主体有权标记失效；目标可识别（记录、对象或载荷摘要之一）。

1 在同一事务内：写入失效标记记录，推进 workspace_epoch.validation_epoch，向 invalidation_queue 插入根项。

2 失效传播工作者（M3）按反向依赖闭包标记 invalidation_mark，将受影响的 Claim、TaskRevision 前提、ContextManifest 置入待重评（第 7.3 节算法）。

3 关键读取与动作准入发现投影落后（epoch 或 applied_sequence 落后）时执行闸门：等待、对依赖闭包重验，或返回需要重评（E401）。普通探索检索可返回显式标注陈旧的候选。

语义边界：撤回支持不是新增反证；无法访问证据不等于从未有过证据；全局 epoch 只解决已收到的更新，外部世界尚未通知的变化依赖过期策略与主动复查，不属于本操作承诺。事务边界：标记与入队单事务，传播本身异步但带水位。

5.5 prepare_effect：意图准备

代码 12 prepare_effect 签名

```
prepare_effect(ws, branch, expected_head, request, principal,
               intent: ActionIntent 载荷, grant_id, reservations: [{resource_key,
amount}])
    -> {intent_id, outbox_id, commit_id, snapshot}
```

前置条件：意图冻结（operation_digest 已按 9.1 节计算）；授权合格（Grant 活跃、operation_digest 匹配、目标在 allowed_destinations 内、未过期未撤销未用尽）；资源可预留；关键证据与 validation_epoch 校验通过（precondition_refs 的闭包无失效标记）。完整伪代码由研究设计报告附录 A 给出，本节为其工程化步骤。

1 开启可串行化事务：验证请求身份与冻结摘要一致。

2 验证分支头与声明读版本（expected_head 及 precondition 声明的版本仍为当前认知基线）。

3 验证当前策略与签发者授权（策略服务检查点 CP-2，第 9.2 节）。

4 验证关键证据与验证纪元（闭包内无 invalidation_mark，workspace_epoch.validation_epoch 已被传播水位覆盖）。

5 原子预留预算并占用授权次数（第 7.5 节；次数按逻辑意图计量，占用于 grant_usage）。

6 追加 ActionIntent（PROPOSED 转 PREPARED）与 Reservation 记录。

7 同事务插入 effect_outbox 条目（PENDING），提交事务。

失败语义：任一步骤失败整体回滚，不留半准备状态。错误码：E401 前提失效待重评、E402 授权不匹配、E403 授权过期或撤销、E501 资源不足、E201 基线冲突。

5.6 dispatch / execute_outbox：网关执行

代码 13 execute_outbox 签名

```
execute_outbox(gateway_principal) -> 循环处理；单条流程：
  claim(intent_id) -> revalidate(intent_id) -> record_attempt_start
  -> call_connector(frozen_args, contract_key)
  -> persist(observation, receipt) -> settle_or_retain(intent_id)
```

前置条件：网关以可信身份运行；outbox 条目可领取（state 为 PENDING，或 CLAIMED 且租约到期）。领取使用 SELECT FOR UPDATE SKIP LOCKED，多实例安全。

1 原子领取并重新验证当前条件：授权仍有效、前置版本未失效、删除策略未命中、预留仍持有（与准备时不同的检查点：此刻的最新性）。

2 在实际尝试前追加 attempt-started 记录（先记录、后调用；崩溃窗口因此可被识别为 UNKNOWN，这是保守但诚实的表示）。

3 以冻结参数与契约键调用连接器；调用在数据库事务外进行。

4 将响应保存为新 Observation，再写 Receipt（分类依据必须引用连接器契约条款）。

5 推进效果状态：任一尝试确认已生效则结算预留（committed）；全部可能送达的尝试均确认未生效且无在途或未知尝试则释放或按策略重试（released）；否则置 OUTCOME_UNKNOWN 并保留预留（uncertain）。

崩溃恢复：租约到期只说明本地持有者失联，不证明远端未执行；到期条目重新领取时先对账（第 7.7 节），不盲目重发。已确认成功不得被迟到的失败或未知

回执覆盖；重试回执冲突时保留冲突记录并对账（E605）。错误码：E601 连接器契约缺失、E602 分类依据不成立、E604 无幂等契约禁止自动重发。

5.7 resolve_effect：对账与结算

代码 14 resolve_effect 签名

```
resolve_effect(ws, intent_id, principal,
               basis: {receipt_id | query_result | manual_review})
    -> {effect_state, reservation_settlement}
```

前置条件：存在可核查回执或对账依据。查询类依据仅在连接器契约保证查询覆盖范围、可见性与最终性时可采信为未生效证据；最终一致索引中暂时不存在仍判 UNKNOWN（E606）。

1 按 Attempt 聚合证据，逐项核对 PARTIAL 明细。

2 满足结算守卫（第 6.4 节）后推进效果状态并结算预留：成功转 committed；确认未生效转 released；未知转 uncertain 并保留。

3 一次尝试确认未生效不足以释放整个意图的预留；只有所有可能送达的尝试均可靠确认未生效且无在途或未知尝试，才判定意图未生效。

5.8 project_context：上下文投影

代码 15 project_context 签名

```
project_context(ws, branch, task_revision_id?, query,
                budget: {token_limit}, as_of?, valid_at?)
    -> {context_manifest_id, rendered_input_ref, omitted_items_summary}
```

前置条件：当前访问策略允许（policy_epoch 与水位闸门检查）。处理流程按第 7.4 节算法：候选检索（可带陈旧标注）、访问与适用性过滤、绑定单元组装、必需约束（M）优先装配、效用排序（U）填充预算、截断记录进 omitted_items_summary。必需约束超出预算时返回 E701，调用方必须分阶段读取、提高预算或暂停动作，不得静默丢弃关键约束。排序只能作用于已通过访问与适用性过滤的候选。LLM 的数值汇总、唯一性计数和状态迁移应调用确定性账本查询（第 7.4 节步骤 6），不让模型从片段心算。

5.9 retire_or_purge：撤回与删除

代码 16 retire_or_purge 签名

```
retire_or_purge(ws, request, principal,
               scope: {record_ids | object_ids | payload_digests | label_selector},
               mode: {retire | purge})
    -> {tombstone_ids, derived_regeneration_plan}
```

前置条件：删除范围已解析（scope 可枚举）；principal 具备该 policy_domain 的删除权限。retire 仅标记失效（走 5.4 节）；purge 按第 7.8 节删除时序执行：墓碑先行、在线清理、派生内容再生、备份与恢复路径门控。保留独立证据支持某结论与保留含已删秘密的旧结论文本是两个问题：正文的删除域处理不因存在独立支持而豁免；历史回放在删除后可能返回输入不可恢复，这是明确的能力边界。错误码：E801 范围不可解析、E802 复活阻止、E803 策略禁止删除。

5.10 辅助接口

表 25 辅助接口一览

接口	签名要点	语义与限制
fork	fork(ws, snapshot) -> branch_id	建立分支根并共享不可变记录；新增记录仅本分支可见；分支用于比较方案，不复制现实
merge	merge(ws, branch_a, branch_b, resolution) -> commit_id	生成新记录说明采用哪些版本、哪些冲突未解决；语义合并不得由字符串拼接或最近时间戳代替；预算、审批次数与任务唯一完成等约束不可取并集
replay	replay(ws, branch, upto_sequence) -> 重建投影	确定性 reducer 按序应用；只输入已记录的模型输出、回执与审批结果，不重新调用模型或外部工具；载荷已删除时只恢复可保留状态
reconcile	reconcile(ws, intent_id) -> 对账结果	按幂等键或外部作业 ID 查询远端；信任边界见 5.7 节
query_graph	query_graph(ws, as_of, valid_at, filters) -> 只读视图	所有读取执行策略检查与存在性隐藏；对无权主体连证据存在性也按策略控制
submit_proposal	统一的提案入口	智能体只能经此提交建议；类型检查与权限检查由代码执行，建议不等于签发

6 状态机详细设计

任务与行动需要不同状态机：任务状态描述工作进展，效果状态描述外部世界的可见结果，预留状态描述资源占用，授权状态描述许可的有效性。四类状态机相互独立又通过事件关联：失效传播使任务进入待重评，回执推进效果状态，效果状态驱动预留结算，网关在每次领取时重验证授权。

6.1 效果状态机 EffectState

七态外加 PARTIAL 是外部效果的最小状态集：通用 FAILED 不足以处理部分成功与错误码不明确的服务。转移表如下，守卫不满足时转移被拒绝并记录 unresolved_reason。

表 26 EffectState 转移表

当前状态	事件	守卫条件	下一状态	动作
PROPOSED	prepare_effect 提交	授权、资源、前提、纪元全部通过（5.5 节）	PREPARED	写 intent 与 outbox，原子预留
PREPARED	网关领取并重验证通过	授权仍有效、前提未失效、预留持有	IN_FLIGHT	先写 attempt-started 再外呼
PREPARED	撤销请求	尚未发出任何尝试（outbox 未领取）	CANCELLED_BEFORE_DISPATCH	释放预留，回收授权占用
IN_FLIGHT	收到回执：确认生效	分类依据成立（契约条款）	CONFIRMED_SUCCEEDED	预留转 committed
IN_FLIGHT	收到回执：确认未生效	该尝试可靠确认；且无其他在途/未知尝试	CONFIRMED_NOT_APPLIED	按策略重试（回 PREPARED）或释放预留
IN_FLIGHT	超时且无法判定	无回执或回执不可采信	OUTCOME_UNKNOWN	预留转 uncertain 并保留，进入对账
OUTCOME_UNKNOWN	对账确认生效	查询契约满足覆盖/可见性/最终性，或收到迟到回执	CONFIRMED_SUCCEEDED	结算 committed
OUTCOME_UNKNOWN	对账确认未生效	同上且所有可能送达尝试均确认未生效	CONFIRMED_NOT_APPLIED	释放或按策略重试
IN_FLIGHT / OUTCOME_UNKNOWN	部分生效回执	复合操作已拆分、逐项可核查	PARTIAL	逐项对账，逐项结算

当前状态	事件	守卫条件	下一状态	动作
CONFIRMED_SUCCEEDED	迟到回执冲突	任何已确认成功不得被覆盖	保持并记录冲突	保留冲突证据，转人工或规则对账（E605）

回执先属于具体 Attempt，再按 Intent 聚合：一次尝试未生效不释放整个意图的预留；已确认成功不被迟到回执覆盖。

6.2 尝试状态机与幂等契约

表 27 AttemptStatus 转移表

当前状态	事件	下一状态	说明
（新建）	网关记录后外呼	STARTED	先落库再调用，崩溃窗口可识别
STARTED	响应持久化为 Observation + Receipt	RESPONSE_PERSISTED	分类依据随回执保存
STARTED	超时或连接失败	NO_RESPONSE	尝试级终态；意图层是否 UNKNOWN 由聚合规则决定

表 28 幂等契约 IdempotencyContract 结构

字段	取值与规则
mode	KEYED：远端支持幂等键，重试沿用同一逻辑意图、同一规范化参数与同一键；QUERY_ONLY：远端可查询回执但对账受信任边界约束；NONE：无幂等承诺，默认暂停自动重发（E604）
key / key_scope / key_expiry	键值、其服务范围（命名空间）与有效期；键已过期、参数改变或服务无承诺时，不得继续声称重试安全（E603）
retry_policy	重试次数上限、退避策略、是否允许跨网关实例重试
conditional_update	远端是否支持 If-Match、版本前提或事务；强条件更新依赖远端能力，HTTP 幂等语义不可外推到任意工具 POST [S20]

6.3 资源预留状态机 ReservationState

表 29 ReservationState 转移表

当前状态	事件	守卫条件	下一状态
（新建）	prepare_effect 原子预留	账本余量充足（I5）	reserved

当前状态	事件	守卫条件	下一状态
reserved	意图进入 IN_FLIGHT 后无法判定	效果状态转 OUTCOME_UNKNOWN	uncertain
uncertain	对账确认生效	结算依据可核查	committed
uncertain	对账确认未生效	全部可能送达尝试均确认未生效且无在途/未知尝试	released
reserved	意图确认未生效或取消	同上释放守卫；UNKNOWN 期间禁止释放	released
reserved	意图确认成功	验收前即可结算	committed

UNKNOWN 必须保留资源和授权次数的占用，不能随租约到期释放：否则第一次请求实际已成功却仍未知时，第二个动作会重复消耗同一预算。租约到期只说明本地持有者失联，不证明远端未执行 [S12]。

6.4 授权生命周期 GrantStatus

表 30 GrantStatus 转移表

当前状态	事件	守卫条件	下一状态
（签发）	允许的签发者创建 Grant	operation_digest、scope、expiry、use_limit 完整；委派收敛于父授权	active
active	dispatch 重验证	now 小于 expiry 且未撤销且 uses_consumed 小于 use_limit	保持 active
active	时钟越过 expiry	由网关重验证判定	expired
active	撤销调用 revocation_handle	撤销立即影响后续 dispatch；已发出的外部请求无法收回	revoked
active	占用次数达到上限	按逻辑意图计量；重试不新增占用	exhausted

6.5 任务状态机 TaskStatus

表 31 TaskStatus 转移表

当前状态	事件	守卫条件	下一状态
DRAFT	任务修订提交且验收谓词完整	acceptance_refs 非空	ACTIVE

当前状态	事件	守卫条件	下一状态
ACTIVE	失效传播命中任务前提或前置任务	invalidation_mark 命中 depends_on 或 precondition 闭包	PENDING_REEVALUATION
PENDING_REEVALUATION	重评完成且前提恢复	受影响前提重验通过或以新修订显式接受变化	ACTIVE
ACTIVE	验收判定	全部验收谓词通过（收到成功回执不等于验收通过）	COMPLETED
ACTIVE / PENDING_REEVALUATION	取消	未完成的外部意图按各自状态机处理，不因取消而虚构结果	CANCELLED

7 核心算法设计

本章给出九个算法的伪代码。伪代码为接口语义描述，采用类 Pascal 风格，不含部署细节；连接器契约、事务隔离级别与错误处理按第 4、5、6 章执行。所有算法不得在持有数据库锁期间调用模型或外部工具。

7.1 串行提交协议

代码 17 串行提交协议

```
procedure commit_proposal(ws, branch, proposal, expected_head):
  begin serializable transaction
    if exists(commit where ws, branch, request_id = proposal.request_id) then
      if old.request_digest != proposal.request_digest:
        reject E202  -- 一键两用
      else return old_result  -- 幂等重放
    head := read head_sequence(ws, branch) FOR UPDATE
    if head != expected_head: reject E201  -- 冲突重新检查，非静默合并
    validate schema(proposal.records)              -- 按 kind 注册表
    validate refs: 目标已存在或同批前序；无未来引用（I2）
    validate 批内拓扑序（DERIVATION 边构成 DAG）
    seq := head + 1
    insert commit(..., sequence=seq, reducer_version=RV)
    insert records with recorded_at = service_clock()
    insert record_ref edges
    update object_heads conditionally (head_sequence < seq)  -- I4
    enqueue invalidation roots if any RETIRE marks
  commit
  notify index workers; return snapshot(seq, epochs)
```

同一工作空间内按序号串行，多工作空间并行。Agent 在确定快照上产生提案，提交时携带 expected_head；冲突时调用方在最新快照上重新检查依赖后重提。该选择简化一致性证明，代价是热点空间写入吞吐受限，扩展路径见第 11 章。

7.2 支持电路求值

代码 18 支持电路求值

```
function evaluate_support(claim, as_of, policy) ->
    {four_state, supporting_paths, refuting_paths, truncated}:
  j := latest Justification targeting claim where j.sequence <= as_of
  eligible(ref) :=
    ref 可读(policy) and ref.sequence <= as_of
    and 无 invalidation_mark 命中 ref 且 mark.epoch 未被传播水位覆盖
    and ref 的 freshness/visibility/retention 检查通过
  s := false; r := false; memo := {}
  for group in j.support_groups:            -- OR of AND
    if all(eligible(e) for e in group):
      s := true; record path(group); break
  for group in j.refute_groups:
    if all(eligible(e) for e in group):
      r := true; record path(group); break
  propagate rule_refs: 规则失效 => 其所在组全部失效（递归 memo 共享子表达式）
  budget := {max_paths, max_depth, max_recompute}
  if budget exhausted: truncated := true  -- 返回需要进一步评估，不谎称完整
  four_state := (s, r) 映射：NONE / SUPPORTED / REFUTED / CONTESTED
```

求值继承真值维护与数据溯源思想，不是新的概率模型。最小支持集合的完整展开可能指数增长，因此电路采用共享子表达式的 AND/OR 结构并设置路径数、深度与重算预算；超限返回 truncated，调用方按需要进一步评估处理。CONTESTED 结果必须向读取方展示或进入裁决，不得被静默择一。

7.3 失效传播与读取闸门

代码 19 失效传播与读取闸门

```
procedure invalidate(target, reason):           -- 事务部分
  begin transaction
    insert retire/purge mark record
    epoch := workspace_epoch.validation_epoch + 1
    update workspace_epoch set validation_epoch = epoch
    insert invalidation_queue(root=target, epoch)
  commit

procedure propagate(worker):                    -- 异步部分
  item := claim next PENDING queue item
  closure := reverse_deps(root=item.root)        -- record_ref 反向闭包
  for each hit in closure:
```

```
    insert invalidation_mark(record=hit, epoch)
    if hit 是 Claim: 标记支持电路待重算
    if hit 是 TaskRevision 前提: task_state -> PENDING_REEVALUATION
    if hit 是 ContextManifest: 标记清单过期
  update index_watermark(applied_sequence, epoch)

function read_gate(ws, needed_deps, purpose):    -- 关键读取/动作准入闸门
  if watermark.epoch < ws.validation_epoch
     or any mark 命中 needed_deps 未处理:
    case purpose:
      critical: wait or 对依赖闭包同步重验；仍落后 -> E401 需要重评
      exploratory: 返回候选并显式标注 stale
```

异步索引必须带 applied_sequence 与 epoch；失效传播未完成时，旧索引命中不得当作当前已验证事实。后续优化可用细粒度依赖戳缩小无关更正造成的阻塞，第一版以闭包重验保证正确性。全局 epoch 只解决已收到的更新，不知道外部世界尚未通知的变化。

7.4 上下文投影

代码 20 上下文投影

```
procedure project_context(ws, task, query, budget):
  check read_gate(purpose=critical if 高影响动作 else exploratory)
  C := retrieve_candidates(query, watermark)      -- 可含 stale 标注
  C := C filtered by 访问策略(CP-4) 与适用性(scope, valid_at, as_of)
  bind := for c in C: 组装绑定单元(正文片段 + 状态 + 适用范围
          + 关键来源指针 + 未解决冲突 + 否定条件 + 单位 + 时间 + 标签)
  M := bind 中标记为必需约束/关键证据/风险提示的单元
  cost(M) := tokenize(M, tokenizer_version)
  if cost(M) > budget.token_limit:
    return E701  -- 分阶段读取 / 提高预算 / 暂停动作，禁止静默裁剪
  U := rank_by_utility(bind - M)                  -- 效用为待验证估计
  fill remaining budget with U 顺序装入
  rendered := render(M + selected, order, transforms)
  persist ContextManifest(selected_items, rendered_digest,
         omitted_summary, labels := 并集(inherited_data_labels))
  数值汇总/计数/状态迁移: 调用确定性账本查询，不经模型心算
  return manifest
```

摘要只是派生内容，继续指向原文；不得因摘要更短而丢掉否定条件、单位、时间或权限标签。优化问题只规定目标（预算内最大化效用并保全部必需约束），不宣称已有精确最优算法。

7.5 原子预留与结算

代码 21 原子预留与结算

```
procedure reserve(ws, intent_id, asks):  -- asks: [{resource, amount}]
```

```
  begin serializable transaction
    for a in asks sorted by resource_key:
      l := SELECT * FROM budget_ledger WHERE resource_key = a.resource
             FOR UPDATE
      if l.limit - l.reserved - l.committed < a.amount:
        reject E501(可解释余额与占用明细)
      UPDATE budget_ledger SET reserved = reserved + a.amount
      insert reservation(state=reserved, intent_id, amount)
    consume grant_usage(grant, intent_id)  -- 次数按逻辑意图计量
  commit

procedure settle(reservation, outcome):
  begin transaction
    if outcome = SUCCEEDED: reserved->committed
    elif outcome = NOT_APPLIED and 释放守卫成立: reserved->released
    elif outcome = UNKNOWN: reserved->uncertain  -- 保留占用
  commit
```

预算 100、两个 Agent 各读到 100 并各建消耗 60 的动作时，逐对象 CAS 会全部成功并超额；正确做法是本算法的账本行锁加余量判断：第一个成功后余额 40，第二个被拒绝并可解释返回。稳定快照不等于可串行化 [S19]。

7.6 outbox 执行循环

代码 22 outbox 执行循环

```
procedure execute_outbox(gateway):
  loop:
    item := SELECT .. FROM effect_outbox
            WHERE state='PENDING' OR (state='CLAIMED' AND lease_expired)
            ORDER BY next_retry_at FOR UPDATE SKIP LOCKED LIMIT 1
    if none: sleep(backoff); continue
    claim(item, lease=now+T)                          -- 原子领取
    begin transaction
      revalidate: grant 活跃且摘要匹配且目标未变;
        precondition 闭包无失效标记; 删除策略未命中; 预留仍持有
      if revalidate 失败: state -> ABANDONED; 释放; continue
      append attempt(STARTED, dispatch_digest)      -- 先记录后外呼
    commit
    resp := call_connector(frozen_args, contract_key)   -- 事务外
    begin transaction
      obs := persist Observation(resp)
      receipt := persist Receipt(分类依据=契约条款, evidence=obs)
      aggregate: 按 6.1 节规则推进 effect_state
      settle_or_retain reservation
      state -> DONE or 重排 next_retry_at
    commit
```

本地可证明的边界是网关准入的线性化时刻：外部请求一旦发出，后来的撤销无法收回；远端资源在检查与使用之间变化，不能靠本地事务消除。强条件更新需远端支持 If-Match、版本前提或同等机制 [S20]。

7.7 幂等与对账

代码 23 对账算法

```
procedure reconcile(ws, intent_id):
  c := connector_contract(intent.connector)
  if c.mode = KEYED and key 未过期 and 参数摘要一致:
    r := query(idempotency_key)  -- 远端按键返回原结果
  elif c.supports_job_query and attempt.external_job_id 存在:
    r := query(external_job_id)
  else: 保持 UNKNOWN; 通知人工或策略通道
  采信条件: 契约保证查询覆盖范围、可见性与最终性;
    最终一致索引中暂时不存在 => 仍为 UNKNOWN（E606）
  if r 冲突已有 CONFIRMED_SUCCEEDED: 保留冲突并记录（E605），不覆盖
  resolve_effect(intent_id, basis=r)
```

既不能查询又不幂等的操作，默认暂停自动重发（E604）。查询未找到只有在连接器保证查询覆盖范围、可见性与最终性时，才能作为未生效依据。

7.8 删除传播与复活阻止

代码 24 删除传播时序

```
procedure purge(scope):
  resolve scope -> {objects, records, payload_digests, labels}
  begin transaction            -- 墓碑先行
    insert tombstone(scope, policy_epoch = next)
    advance workspace_epoch.policy_epoch
    invalidate(scope, reason=purged)   -- 走 7.3 节传播
  commit
  cleanup_online: blob_object -> pending_purge -> purged;
    清理索引条目、缓存、渲染输入副本
  regenerate_derived: 仍获准使用的证据重新生成干净派生内容;
    不得只给原文加一句来源已删除
  gate_recovery: 回放与备份恢复路径读取前检查墓碑;
    受禁内容不因旧快照或备份重新可见（I10）
  边界: 软删除/删除主副本/销毁一把密钥不足以证明备份、第三方副本、
    已输出信息全部消失; 记录 ID、关系边、时间与哈希也可能泄露信息
```

7.9 确定性回放恢复

代码 25 确定性回放

```
procedure replay(ws, branch, upto_sequence):
  state := empty_projection(reducer_version 链)
  for c in commits(ws, branch) where sequence <= upto_sequence:
    inputs := records(c) 的已记录模型输出、工具回执与审批结果
    state := apply(reducer(c.reducer_version), state, select_b(inputs))
    -- 不调用模型; 不调用外部工具; 同种子也不承诺云模型输出相同
  for d in 已删除载荷(closed by tombstone):
    保留结构与摘要, 正文标记不可恢复   -- 只能恢复可保留状态
  rebuild derived indexes from state (带水位回填)
```

```
    return state
```

重新向模型提问属于新运行，不属于回放。回放的正确性依赖 reducer 固定版本与输入完整性；这也是 record 不可变触发器存在的理由。

8 并发控制与一致性设计

8.1 工作空间内有序提交

MVP 为每个工作空间维护单一提交序列（7.1 节协议）：提交串行、序号无空洞、多工作空间并行。智能体读取稳定快照后在事务外思考与调用模型，提案提交时才做版本与资源检查——不在模型推理期间持有数据库锁。冲突处理策略是显式重新检查：expected_head 不匹配即拒绝并返回当前头，由调用方决定合并方式，不做静默最后写入获胜。

8.2 事务边界一览

表 32 事务边界表

操作	事务内写集	锁窗口	事务外动作
commit_proposal	commit、record、record_ref、object_heads、invalidation_queue	短；仅行锁	索引通知
prepare_effect	intent 记录、effect_state、reservation、grant_usage、effect_outbox	短；可串行化	无（摘要计算在事务前）
execute_outbox 领取	outbox 领取、attempt(STARTED)	短；SKIP LOCKED	连接器外呼
execute_outbox 回执	Observation、Receipt、effect_state、reservation 结算	短	无
invalidate	失效标记、epoch、队列	短	异步传播（7.3 节）
purge	tombstone、policy_epoch	短	在线清理、派生再生、备份门控
project_context	ContextManifest 写入	短	检索、排序、渲染、模型调用
模型推理	不开启事务	无	全部推理与工具预取

8.3 预算账本与写偏斜防范

共享预算、库存与授权次数必须作为同一事务中的约束处理。研究设计报告给出的反例：预算 100，两个 Agent 各读到 100 并各建消耗 60 的动作，逐对象 CAS 全部成功也会超额。防范措施：预留一律走 7.5 节算法（账本行 FOR UPDATE 加余量判断），隔离级别取可串行化；快照隔离下的写偏斜由谓词锁兜底 [S19]。扩展到细粒度并发后，须显式处理多对象读集、谓词条件、写偏斜与序列化失败重试，重试不消耗 request_id 幂等配额。

8.4 分支 fork 与 merge 协议

fork(snapshot) 建立分支根并共享不可变记录，新增记录仅对该分支可见；分支用于比较研究假设或候选计划。merge 生成新记录，说明采用哪些版本、哪些冲突仍未解决；语义合并不得由字符串拼接或最近时间戳代替。可合并性规则：只追加独立观察的集合可采用可合并集合思路；预算余额、审批次数与任务唯一完成等约束不能取并集，必须回到工作空间账本重新校验。外部已经发生的效果属于现实执行账本，不能通过回滚分支撤销；从分支发起实际效果仍须通过公共网关与公共资源预留。跨工作空间原子事务不属于 MVP 承诺。

8.5 不变量到机制的映射

表 33 不变量实现映射（I1–I10）

编号	不变量	实现机制	验证方法
I1	正常修订只追加，版本引用不悄悄改指向	record 不可变触发器；可信写入服务	TC-INV-01 随机 UPDATE/DELETE 注入
I2	不引用未来记录	写入器验证 ID 与批内拓扑序（7.1 节）	TC-INV-02 批内乱序与跨批未来引用
I3	同一 request_id 不接受不同内容	UNIQUE(workspace, request_id) + 摘要比对	TC-INV-03 一键两用并发
I4	旧版本写入不能静默覆盖当前版本	object_heads 条件更新 + expected_head 检查	TC-INV-04 并发 revise 竞争
I5	预留加已确认消耗不超额度	可串行化事务 + 账本 CHECK（7.5 节）	TC-05 双预留 60/预算 100
I6	实际动作与仍有效授权的具体内容匹配	网关摘要匹配重验证（9.1、7.6 节）	TC-04 审批后改参数

编号	不变量	实现机制	验证方法
I7	超时不能直接推导未生效	UNKNOWN 状态 + 对账信任边界（7.7 节）	TC-06 回执前崩溃；TC-07 租约过期
I8	失效投影不能作为当前已验证前提	epoch 闸门与闭包重验（7.3 节）	TC-10 索引落后且前提被更正
I9	回放不重新调用外部效果	确定性 reducer 与记录结果（7.9 节）	TC-INV-05 回放比对投影
I10	当前禁止访问或删除不因旧快照失效	墓碑门控全部读取与恢复路径（7.8 节）	TC-08 删除后恢复旧快照

可采用归纳证明维持不变量：初始空状态满足；每个允许操作在提交前检查前置条件并在一个事务内更新记录、资源与必需投影；因而每个提交后的状态继续满足结构性性质。证明范围不含：未知依赖、错误抽取、绕过网关、恶意数据库管理员、不遵守契约的外部服务；数据结构无法证明任意自然语言结论为真。

9 授权、隐私与删除设计

9.1 operation_digest 规范化规则

授权绑定具体操作内容，而非操作名称。规范化采用 RFC 8785 JSON Canonicalization Scheme 思路并固定 canonicalization_version；大整数、单位、时间格式与二进制文件摘要按下列附加规则编码 [S21]。摘要覆盖字段：operation、target_ids、executor_principal、connector_identity 与 connector_version、arguments（规范化后）、artifact_revision_ids 对应的确切内容摘要。规范化版本变化时旧摘要不重算，网关按 Grant 内记录的 canonicalization_version 选择校验器。

表 34 规范化附加编码规则

数据类别	编码规则
大整数	十进制字符串，禁用浮点与科学计数法
时间	RFC 3339 UTC，同一精度
二进制与文件	sha-256 摘要加字节长度，不内联内容
计量与金额	数值字符串加显式单位字段，单位参与摘要
数组与对象键	JCS 排序规则；数组保持语义序（顺序敏感时先标注）

数据类别	编码规则
枚举	注册表中的规范名，禁止同义词

9.2 权限检查点

权限检查同时覆盖记录、关系边、搜索候选、缓存、全文提示与产物。检查点在代码中集中声明，禁止在业务分支内散落实现。

表 35 权限检查点清单

检查点	位置	检查内容
CP-1 提案准入	M1 submit_proposal	principal 可写该 kind 与 policy_domain；Grant/Receipt/回执类拒绝非可信签发者
CP-2 授权签发与使用	M4 + 网关	签发者权限；委派收敛（子范围不超父）；使用时摘要匹配、目的、目标、有效期、次数
CP-3 记录读取	M3 query_graph	Visibility 解析；对无权主体连证据存在性也按策略隐藏
CP-4 检索候选过滤	M3/M6 project_context	候选先过访问与适用性过滤，排序只作用于已过滤集合
CP-5 渲染与缓存	M3 渲染缓存	缓存键含 policy_epoch；渲染输入的复用不得跨越策略纪元
CP-6 产物披露	M3/M2 产物下载	读取权限与向某目标披露数据的权限分开判定
CP-7 名称解析	M4 授权前	人名、服务器名等解析为稳定身份并冻结；执行时不得悄悄重新指向另一对象 [S15]

来自网页的提示注入仍可能影响模型行为，但不会因此获得签发 Grant 或执行未经许可操作的能力——该边界依赖所有外部效果都经过网关（第 2.3 节信任边界）。

9.3 数据标签传播

数据标签沿实际使用的输入传播：Observation 标签进入 ContextManifest 的 inherited_data_labels 并集，随模型输出与产物继续传播（Artifact.data_labels 只增不减）。读取敏感数据后生成的派生内容不能自动成为公开信息；披露或降级需要显式规则（策略服务的降级规则表，阶段 B 提供最小集：仅允许向更高或相等标签域流动）。

9.4 删除域与能力边界

不可变指正常业务修订不能覆盖旧版本，不代表正文永远不可删除。删除域覆盖：正文、嵌入向量、摘要、完整渲染输入、附件、缓存与必要元数据。删除时序按 7.8 节执行，墓碑先于一切清理持久化。能力边界必须向产品与用户明示：仅做软删除、删除主副本或销毁一把密钥，不能自动证明备份、第三方副本及已经输出的信息全部消失；记录 ID、关系边、时间与哈希也可能泄露信息；普通权限隔离不等于差分隐私，若需要多租户统计或共享记忆的差分隐私保证，应另行定义邻接关系、发布机制与隐私预算，不能从本数据结构推出。

9.5 标识与摘要暴露控制

ID 为租户范围内不可猜测标识（UUIDv7/ULID），枚举攻击不可行。哈希用于完整性与内容绑定，不公开暴露敏感低熵内容的摘要（低熵内容可在摘要上离线穷举）；对外查询接口返回的定位符一律经能力令牌包装，且能力令牌受 policy_epoch 失效约束。

10 错误处理设计

错误码全局唯一、分层分配。返回结构统一为 {error_code, message, retryable, details}：retryable 表示调用方可以原 request_id 安全重试（幂等键仍有效）；details 携带可解释信息（如预算余额与占用明细、当前 head、命中的墓碑）。下表为第一版全量目录。

表 36 错误码目录

码	名称	语义	调用方处理
E101	SCHEMA_INVALID	载荷不符合注册表 schema	修正后重提
E102	SCHEMA_VERSION	schema 版本不可解释	升级写入器
E103	KIND_NOT_ALLOWED	principal 无权写入该 kind	转人工或放弃
E201	HEAD_CONFLICT	expected_head 不匹配	读新快照重检查后重提
E202	REQUEST_ID_REUSE	request_id 重复且摘要不同	换新请求；一键两用属违规
E203	REVISE_BASELINE	修订基线不匹配	以返回的当前 head 重做

码	名称	语义	调用方处理
E301	REF_NOT_FOUND	引用版本不存在	修正引用
E302	FUTURE_REF	引用未来记录（I2）	修正批内拓扑
E303	REF_NOT_READABLE	无引用目标读取权限	放弃或申请权限
E401	STALE_PREREQ	前提失效待重评（闸门）	等待或闭包重验后重提
E402	GRANT_MISMATCH	operation_digest 不匹配	重新申请授权
E403	GRANT_INVALID	授权过期、撤销或用尽	重新申请
E501	BUDGET_EXCEEDED	资源余量不足	调整数量或等待释放
E502	RESERVATION_LOST	预留已不存在或被结算	重新 prepare
E601	CONNECTOR_CONTRACT_MISSING	连接器未声明所需契约	补契约或降级人工
E602	CLASSIFICATION_UNSUPPORTED	回执分类无契约依据	置 UNKNOWN 并对账
E603	IDEMPOTENCY_KEY_EXPIRED	幂等键过期或参数改变	不得声称重试安全
E604	RETRY_FORBIDDEN	无幂等契约的自动重发	暂停并转策略通道
E605	RECEIPT_CONFLICT	迟到回执与已确认成功冲突	保留冲突并对账
E606	QUERY_NOT_FINAL	查询不满足覆盖/可见性/最终性	维持 UNKNOWN
E701	CONTEXT_BUDGET	必需约束超出 token 预算	分阶段/提额/暂停动作
E702	CONTEXT_EXPIRED	清单超期或策略纪元变更	重新投影
E801	PURGE_SCOPE_UNRESOLVED	删除范围无法解析	显式列举范围
E802	REVIVAL_BLOCKED	命中墓碑（复活阻止）	放弃该路径
E803	PURGE_FORBIDDEN	策略禁止删除该域	转策略审批

11 性能与复杂度设计

11.1 操作复杂度（成立条件必须随行）

下表沿用研究设计报告第 12.2 节的结论：复杂度必须注明成立条件，不允许无条件的性能承诺。

表 37 操作复杂度

操作	典型代价	不能忽略的条件
按 ID 读取	B 树索引 O(log N)	载荷大小与网络 I/O 另计
追加 k 条引用的记录	约 O((k+1) log(N+E)) + 载荷写入	固定数量 B 树索引；无触发额外失效传播
有界邻域遍历	O(Vr + Er)	r 为实际可达子图规模
失效或删除传播	O(Va + Ea) + 规则重算	最坏可覆盖整图；单条撤回可能触发全图传播
从检查点恢复	增量日志与载荷读取 + reducer 实际工作 + 索引重建	回放不能跳过 reducer 计算
支持电路求值	显式无环电路上线性于电路规模	最小支持集合完整展开可能指数增长，故设预算截断
语义检索与矛盾判断	取决于索引、召回策略与模型	不能统一承诺 O(log N)

11.2 瓶颈与对策

表 38 预期瓶颈与对策

瓶颈	表现	第一版对策	扩展路径（阶段 E）
热门实体依赖扇出	失效传播放大、读放大	闭包规模上限加显式截断标注	细粒度依赖戳
工作空间提交争用	expected_head 冲突率上升	错峰提案、批合并提交	按对象域拆分序列
模型抽取成本	assess 写入延迟高	事务外抽取、异步落账	抽取结果缓存与增量
索引异步延迟	闸门等待变长	优先级队列：critical 依赖先行	并行传播器
删除传播	purge 停顿	墓碑先行后异步清理	域级批量清理

所有异步索引必须带 applied_sequence 与 validation_epoch 水位（4.2 节 index_watermark）；普通探索检索可返回显式标注陈旧的候选，关键读取与动作准入必须过闸门。扩展决策以瓶颈测量证据为前提，先测量再分片、引入专用图存储或压缩历史；压缩不得丢弃仍需审计的版本或未完成动作。

12 测试与验证设计

12.1 三层评估框架

按研究设计报告第 13.1 节，评估分三层隔离变量：第一层用确定性记录与假工具做状态机与并发测试，隔离大模型随机性；第二层把自然语言映射成记录，与 oracle 解析对照，测漏记、误抽取与错误依赖；第三层才做完整智能体任务，测任务成功率与成本。不隔离层次时，指标改善无法归因于数据结构、提示词、更多调用或更强模型。公开基准中 LoCoMo 用于对话长期记忆，LongMemEval 覆盖知识更新与时间推理，StructMemEval 增加树、账本与状态组织压力；外部效果、撤回、授权变化与故障恢复场景由本设计的对抗用例补足 [S8, S22, S23]。

12.2 对抗场景测试用例

下列用例直接对应研究设计报告第 13.3 节的注入事件，是阶段 A、B 的验收测试集。

表 39 对抗场景测试用例

用例	注入	主要观测	通过条件	覆盖
TC-01	旧证据撤回，仍有独立新证据	失效传播与撤销范围	只使失效支持路径退出；独立支持保留	I8, 7.3
TC-02	较新但低可信消息与有效较早证据冲突	冲突处理	不自动覆盖；进入 CONTESTED 展示	7.2
TC-03	两 Agent 各预留 60，总预算 100	资源账本	第二笔拒绝且失败可解释（余额明细）	I5, 7.5
TC-04	审批后修改参数、接收者或产物	操作摘要匹配	新动作不能借用旧授权（E402）	I6, 9.1
TC-05	外部成功后、回执持久化前崩溃	UNKNOWN 恢复	重启后不盲目新建重复逻辑效果	I7, 7.6
TC-06	租约过期且旧 worker 仍在执行	网关与远端去重	有契约阻止重复；无契约时保留边界并告警	I7, 7.7
TC-07	删除后恢复旧快照或备份	删除域与恢复门控	受禁内容不重新可见（E802）	I10, 7.8
TC-08	多个 Agent 转载同一来源	来源谱系	同一根来源谱系；不按独立证据重复计数	7.2, 3.4

用例	注入	主要观测	通过条件	覆盖
TC-09	上下文必需约束超过预算	检索器行为	返回 E701；分阶段或明确不足，不静默裁剪	7.4
TC-10	索引落后且前提刚被更正	水位与执行闸门	陈旧候选不能直接获准执行（E401）	I8, 7.3

表 40 确定性不变量用例

用例	注入	预期
TC-INV-01	对 record 表直接 UPDATE/DELETE	触发器拒绝；审计告警
TC-INV-02	构造批内未来引用与跨批未来引用	写入器拒绝（E302）
TC-INV-03	并发提交相同 request_id 不同摘要	恰好一个成功，另一个 E202
TC-INV-04	并发 revise 同一对象	一个成功，另一个 E203 并收到新 head
TC-INV-05	回放重建投影与在线投影逐字段比对	完全一致（除显式异步缓存）

12.3 指标与门控

表 41 分层指标

层	指标
结构层	失效传播 precision 与 recall、冲突静默覆盖次数、未经有效授权的动作数、重复外部效果数、恢复状态一致率、删除复活率
任务层	任务成功率、正确拒答率、证据支持准确率、人工裁决负担
系统层	p50 与 p95 延迟、每任务总费用、存储增长、索引延迟、热点吞吐

门控规则：确定性不变量在已覆盖故障模型内必须无反例——这是实现门槛，不是对现实零风险的证明。统计门控：先小规模 pilot 估计方差与事件率，再冻结有统计功效的样本量与阈值；若独立同分布测试中 n 次无失败，单侧 95% 失败概率上界为 1 - 0.05^(1/n)，大样本近似 3/n；故障有关联时该界不可直接套用。预注册的有条件目标：在知识更正与故障压力场景显著减少错误动作，在干净场景不明显损失成功率，且额外非模型延迟与 token 成本处于应用可接受区间。具体百分比由目标产品与 pilot 决定，本文不虚构性能结果。

12.4 公平基线与消融

所有方法共享冻结的数据划分、模型标识、工具模拟器与尽可能相同的调用预算；分别报告写入、维护、检索与决策 token，异步预处理也计费；选择多个模型与多个独立任务种子，按场景或任务聚类统计，不把同一轨迹的每个事件当独立样本。基线与消融组合沿用研究设计报告第 13.2 节：消息加摘要、向量或事实记忆、时序图记忆加 checkpoint、成熟工作流加 SQL 版本与 outbox（关键强基线：若其达到同等结果，应接受更简单实现）；消融项为去掉 Justification 分组、去掉上下文清单、去掉 epoch 或内容绑定（仅在隔离测试中运行）。

13 实施计划

实施顺序沿用研究设计报告第 14 章的阶段划分，每阶段给出交付物与进入下一阶段的条件；条件不满足不得携带已知反例进入下一阶段。

表 42 阶段计划

阶段	交付物（本文章节）	进入下一阶段的条件
A 语义内核	记录与提交模型（3、4 章）、串行提交协议（7.1）、预算账本（7.5）、不变量映射（8.5）	I1–I5 无反例；并发用例 TC-INV-01 至 04 与 TC-03 通过
B 执行闭环	授权绑定（9.1）、outbox 与网关（5.6、7.6）、UNKNOWN 对账（7.7）、状态机（6 章）	崩溃窗口与幂等契约用例 TC-05 至 07 通过；I6、I7 无反例
C 证据与上下文	支持分组（3.6、7.2）、失效传播（7.3）、ContextManifest（3.12、7.4）、删除域（7.8、9.4）	TC-01、02、08 至 10 通过；I8、I10 无反例
D 真实任务评估	三层评估与基线（12 章）	证明收益超过成本与简单基线（预注册门控）
E 性能扩展	分片、索引优化、委派与协议适配	已有瓶颈测量证据支持扩展（11.2 节）

产品路径优先完成阶段 A 与 B，再连接模型记忆；研究路径优先在第 11 章约束下实现阶段 C，并直接进入阶段 D 的对抗场景与强基线比较。最值得聚焦的可证伪假设沿研究设计报告的表述：在固定预算下，维护来源到主张再到行动前提的依赖，能否在更正与撤回发生时精确阻止失效行动，同时保留仍有独立支持的工作。若简单 SQL 状态加 checkpoint 与 outbox 的强基线达到同等结果，应接受更简单的实现并收缩 ASG 的适用声明。

附录 A 记录类型注册表

注册表是写入服务 schema 校验的唯一依据。允许签发者栏为空表示任意通过 CP-1 的主体可提议；标注仅可信的 kind 由服务身份强制。新增 kind 必须先修订本表与 reducer，再开放写入。

表 43 记录类型注册表

kind	中文名	载荷定义	允许签发者	允许发出的引用角色
OBSERVATION	观察	3.4 节	任意（经连接器或采集工具）	derived_from, extracted_from, transcribed_from
CLAIM	主张	3.5 节	任意	supersedes, uses_version
JUSTIFICATION	论证	3.6 节	任意	supports, refutes, uses_version（规则引用）
ASSESSMENT	评估	3.7 节	任意（评估者身份入载荷）	uses_version
ENTITY_MERGE	实体合并	3.5 节附注	任意（可撤销）	supersedes（合并决议）
TASK_REVISION	任务修订	3.8 节	任意	supersedes, depends_on, uses_version
ACTION_INTENT	动作意图	3.9 节	任意（提案）	precondition_of, authorized_by, uses_version, selected_into
GRANT	授权	3.10 节	仅被允许的签发者（CP-2）	authorized_by（由意图反向引用）
ATTEMPT	尝试	3.11 节	仅执行网关	attempt_of, produced
RECEIPT	回执	3.11 节	仅执行网关或可信接入	receipt_of
ARTIFACT	产物	3.11 节	网关或受托服务	supersedes（内容版本）
CONTEXT_MANIFEST	上下文清单	3.12 节	读取服务	selected_into（被反向引用）
RULE / VALIDATOR	规则与验证器	3.6 节规则引用	任意（版本化）	supersedes
RETIRE / PURGE_MARK	失效与删除标记	5.4、5.9 节	有权主体	uses_version（指向目标）

附录 B 需求追溯矩阵

研究设计报告第 1 章的九项要求到本设计机制的追溯。逐项可追溯是评审检查表：任何机制变更必须同步检查对应行的测试用例是否仍然覆盖。

表 44 R1–R9 追溯矩阵

要求	机制	本文章节	验证用例
R1 来源可追溯	Observation 与 Claim 引用具体版本；asserted_source 与 authenticated_writer 分离	3.2、3.4、3.5	TC-08
R2 更新不抹去历史	只追加修订链；as_of 与 valid_at 双时间查询	3.3、5.0、7.1	TC-INV-05
R3 冲突可见	独立 Justification；支持/反证分组；CONTESTED 必须展示	3.6、7.2	TC-02
R4 多 Agent 一致性	串行提交 + expected_head；原子资源预留	7.1、7.5、8.1、8.3	TC-03、TC-INV-04
R5 行动可恢复	意图、尝试、回执、结果未知分离的状态机	3.11、6.1、7.6、7.7	TC-05、TC-06
R6 授权不漂移	operation_digest 内容绑定；网关重验证	3.10、9.1、9.2、7.6	TC-04
R7 上下文可解释	ContextManifest 与预算投影；渲染输入摘要	3.12、7.4	TC-09
R8 可撤回可删除	删除域、墓碑先行、恢复门控	7.8、9.4	TC-07
R9 成本可控制	分层索引水位、增量维护、有界查询与预算截断	4.2、7.2、11 章	TC-10

本详细设计说明书与研究设计报告共同构成 ASG 的设计基线。实现过程中发现的偏差应回写为对两份文档的修订，而不是仅在代码中偏离；每次阶段验收以追溯矩阵的用例集为检查清单。